ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Computer Vision
and Geometry Lab

Improving 2D Map-Based Visual Localization

Master Thesis

Alan Savio Paul
Department of Computer Science

Advisors: Paul-Edouard Sarlin, Zador Pataki
Supervisor: Prof. Dr. Marc Pollefeys
Computer Vision and Geometry Group
Department of Computer Science D-INFK

September 20, 2024

Abstract

Typical visual localization methods are designed very differently from the way humans localize themselves on a map - they rely on 3D point cloud maps that are expensive to build and store, especially at scale. Recent works like OrienterNet instead propose a neural network that can localize an image up to 3-DoF on free and publicly available 2D Maps from OpenStreetMap. Since it doesn't require pre-built 3D point clouds, it is much more practical and allows on-device localization in Robotics and Augmented Reality applications.

This thesis focuses on improving the accuracy, efficiency, and scalability of OrienterNet. A key limitation of OrienterNet that we identified is that it uses visual cues from only 32 metres in front of the camera, thus making it incapable of exploiting key information like buildings and roads that lie further away. Trivially extending this range is computationally infeasible, largely due to its high-resolution maps which are essential for accurate localization. In this work, we propose a neural network which jointly learns multi-scale Bird's-Eye-View (BEV) maps from a single image, enabling efficient utilization of visual cues from nearby and further away, ranging up to 128m. Our model learns to optimally combine fine and coarse visual features and maps, which results in increased localization accuracy and additionally also enables highly efficient hierarchical localization on much larger maps.

We demonstrate that our method outperforms OrienterNet by roughly 20% Recall under 20m on maps with a radius of 256 metres. We also propose a variant of our method that is additionally trained on satellite imagery, which contains rich semantic and geometric information. We show that jointly encoding OpenStreetMap and satellite imagery yields richer BEVs and Neural Maps, consequently resulting in an extra 4% improvement.

Acknowledgements

This thesis is the result of many experiences over the past 6 months and throughout my Master's degree. I would like to extend my most sincere gratitude to my supervisor, Paul-Edouard Sarlin, for the valuable feedback, guidance, knowledge, and inspiration throughout my Master's degree. Thank you for the amazing work done in OrienterNet and SNAP, without which this thesis would not have been possible. I would also like to convey my gratitude to my second supervisor, Zador Pataki, for all his support, advice, and spontaneous discussions. Thank you both for sharing your deep insights and encouragement throughout this wonderful journey.

From introducing me to 3D Computer Vision to helping me cultivate a strong passion and expertise in this field, I am deeply grateful to Prof. Marc Pollefeys and his Computer Vision and Geometry lab (CVG) at ETH Zürich. I particularly extend my gratitude towards Rémi Pautrat, Mihai Dusmanu, and Zuria Bauer for supervising my previous projects during my time here. These opportunities, along with this thesis, significantly shaped my approach to research and engineering, and I am very thankful to my supervisors for all their teachings.

I would also like to extend a huge thank you to all my friends who have made this journey thoroughly enjoyable and to my colleagues at the D-Lab for all the interesting discussions and for making it a fun workplace over the last 6 months.

Finally, I would like to express my most heartfelt gratitude towards my mother, father, and sister for their continuous support and encouragement throughout this long journey. This thesis and this Master's degree would not have been possible without them.

Contents

1 Introduction **1**
- 1.1 Contributions . . . 2
- 1.2 Contents . . . 2

2 Related Work **3**
- 2.1 Visual Localization . . . 3
- 2.2 Bird's-Eye-View . . . 4

3 Method **7**
- 3.1 Problem Formulation . . . 7
- 3.2 Method Overview . . . 7
- 3.3 Neural Bird's-Eye-View . . . 9
  - 3.3.1 Image to BEV Mapping . . . 9
- 3.4 Neural Map . . . 10
- 3.5 Multi-Scale Pose Estimation . . . 11
- 3.6 Multi-Scale Training Loss . . . 11

4 Experiments and Evaluation **13**
- 4.1 Experimental Setup . . . 13
- 4.2 Design Decisions: Single-Scale Models . . . 15
  - 4.2.1 Neural BEV: Forward vs Inverse Mapping . . . 15
  - 4.2.2 Neural BEV: Increasing Depth . . . 15
  - 4.2.3 Neural Map Encoding . . . 17
  - 4.2.4 Pose Estimation: Exhaustive vs RANSAC . . . 20
- 4.3 Design Decision: Multi-Scale Models . . . 23
  - 4.3.1 Neural BEV . . . 23
  - 4.3.2 Neural Map Encoding . . . 24
  - 4.3.3 Loss . . . 24
- 4.4 Results . . . 24
- 4.5 Fusing Satellite Imagery . . . 29

5 Discussion and Conclusion **31**
- 5.1 Discussion . . . 31
  - 5.1.1 Limitations . . . 32
  - 5.1.2 Future Work . . . 32

List of Figures

3.1 **Method Overview**. BEV Mapper: We extract features and scales from the input image, which are processed in two multi-scale gravity-aligned 3D grids and vertically max-pooled. The BEVNet then refines the features and outputs a confidence map per BEV. MAP Encoder: We compute two neural maps at different resolutions. During training, they cover areas of different sizes. Matching: We exhaustively match each BEV with its corresponding neural map to produce two probability volumes. The network is trained end-to-end and learns complementary multi-scale features to effectively disambiguate poses on a large map. 8

4.1 **Samples of images and multiscale maps from our training dataset**. 3-DoF pose (x, y, θ) is shown with a red arrow. During training, we use maps with radius 256px, which results in coarser maps covering larger areas. As the map areas grow larger, localizing a single image becomes increasingly more challenging due to the higher number of ambiguous poses. Note how it is much easier for us to localize the images above on the smallest map. 14

4.2 **Forward Mapping vs Inverse Mapping:** Visualizing learned Neural BEV and confidence. In the first two rows, the curved building wall appears to be better represented by the inverse mapping's BEV and its confidence map (blue denotes low confidence). In the second example, the BEVs are similar, with the inverse BEV showing slightly straighter roads. Notice the inverse mapping does not require rectified images, and can thus occasionally benefit from few additional visual cues that are otherwise discarded during rectification. 16

4.3 **Increasing BEV Depth Range yields large improvements**. Chaining results of multiple models trained at different depths and resolutions drastically reduces uncertainty (see sharper likelihood) and results in high accuracy. GT pose (red arrow), estimated pose (black arrow), and the BEV's outline is overlayed on the input map. In this example, the buildings that are far away are crucial for accurate localization. 18

4.4 Similar to 4.3. In this example, localizing with the 32m BEV results in high uncertainty as the visible buildings do not fall within its range. Increasing the range narrows down possible poses. 19

4.5 Example of Query BEV (left) cropped from Neural Map (right) at GT Pose (black arrow). In Section 4.2.4, we evaluate and compare Exhaustive vs RANSAC pose estimation using this BEV. 21

4.6 **Exhaustive Matcher**: Measuring accuracy, time, and memory on varying map sizes and number of rotations 21
4.7 **RANSAC Matcher**: Measuring accuracy, time, and memory on varying map sizes and number of sampled poses 22
4.8 **Visualization of the outputs of the coarse and fine branches**. Row **(a)** shows 3 input test images. Rows **(b)**-**(f)** visualizes the predictions of the BEV mapping module. Row **(g)** contains the PCA plots of the neural map encodings of **(h)**. Row **(i)** shows the pose likelihoods per branch. Note the reduction in pose error when both likelihoods are chained in row **(j)**. 25
4.9 **OrienterNet vs Ours**. Our model learns an optimal combination of coarse and fine features covering larger distances (BEV outlines shown at the predicted pose) than OrienterNet. The examples above show OrienterNet's failures being successfully localized by our model. GT and Predicted poses are shown in red and **black** respectively. 26
4.10 **OSM-only (Failure), OSM+Satellite (Success)**. We see OSM+Satellite successfully localizing these images which OSM-only cannot. This can be attributed to the road markings, trees, poles and other objects that are visible only on the satellite imagery. 27
4.11 **Non-Localizable Failure Cases**. These are examples of images that lack distinctive visual cues that could be used to correctly localize the camera. 28
4.12 **Comparing Neural Maps**. Note the details that emerge when encodings of OSM and Satellite imagery are fused. 29

List of Tables

4.1 Forward Mapping Cost. 16
4.2 Inverse Mapping Cost. 16
4.3 Forward Mapping vs Inverse Mapping Localization Results. 16
4.4 Equal Depth of 32m, Coarser Resolution. We use only the first 32m of the BEV for localization of all 3 models. Resolutions are 0.5mpp, 1mpp, and 2mpp for the 32m, 64m and 128m models respectively. 17
4.5 Exhaustive vs RANSAC matching. We synthesize a perfect BEV by cutting it out of Orienternet's neural map at the GT pose, and evaluate it on our validation dataset. Here, the neural map's radius is 256px. The exhaustive matcher is significantly more accurate and efficient. It scores millions more poses in a fraction of the time taken by the RANSAC matcher, thanks to the highly efficient FFT Convolution. 21
4.6 Localizing single-scale and multi-scale BEVS (Search Radius: 256m). Single-Scale: Increasing BEV depth up to 128m results in increasing recall rates, despite decreasing resolution. Chaining multiple single-scale models yields highest accuracies. Increasing resolution of input map raster is beneficial for coarser models. Multi-Scale: img: sep/shared indicates whether we separate/share decoders for fine and coarse features. map: sep/shared indicates whether we separate/share the encoders - the decoders are always separate. Shared features (2b) outperforms (2a). Uncertainty-weighted loss (2c and 2d) outperform simple loss (2a and 2b). Using separate map encoders with shared image features and separate map encoders consistently score the highest (2e). 22
4.7 Localizing single-scale and multi-scale BEVS (Search Radius: 10m). Single-Scale: Increasing BEV depth ranges up to 128m results in increasing recall rates, despite decreasing resolution. Chaining multiple single-scale models yields highest accuracies. Increasing resolution of input map raster is beneficial for coarser models. Multi-Scale: Naming convention as in 4.6. 23
4.8 Ours vs OrienterNet: Search Radius=256m (Retrieval) 28
4.9 Ours vs OrienterNet: Search Radius=10m (Precision) 28
4.10 Ours vs OrienterNet: Search Radius=48m 28

Chapter 1

Introduction

Visual localization refers to the task of estimating the location and orientation of a query image with respect to a scene representation. This is a classical computer vision problem which has numerous applications in robotics, augmented reality, and autonomous driving. This problem is typically solved using a pipeline with many stages including feature detection, matching, and pose estimation with a 3D point cloud [11, 19, 24, 38, 43]. These methods provide accurate 6-DoF poses but are very expensive to store at scale.

On the other hand, humans are capable of localizing themselves on simple 2D maps by using visual information around them and finding a good match on a map containing sufficient semantic and geometric information such as accurately positioned buildings, roads, and other landmarks. While GPS is cheap and readily available, they often produce noisy and unreliable position estimates, especially in cities with urban canyons.

Several recent approaches aim to localize a camera with 2D maps. The most common type of maps used are satellite imagery [9, 10, 23]. Differently, OrienterNet [34] proposes a method that leverages simpler maps from OpenStreetMap (OSM) [30]. These maps contain simple polygons, lines and points to represent roads, buildings, parks, trees etc. and are freely and widely available online. OrienterNet is capable of localizing a query image by generating a bird's eye view mapping and matching it with a predicted neural map from the OSM raster.

Our work directly builds on top of OrienterNet, with the goal of trying to improve its accuracy. We study design decisions that were introduced in a related work, SNAP [35], and also evaluate other potential areas for improvement. In this thesis, we identify a key limitation of OrienterNet: its BEV representation has a limited depth range of 32m. Thus, it cannot accurately localize images in which most valuable information for localization lies outside this range. Naively increasing the BEV depth range (and consequently the neural map size for matching) is computationally infeasible at the current high resolution. In this thesis, we demonstrate that distant visual information such as the presence of buildings, parks, or even empty space, is highly advantageous for accurate localization, and we propose a method to ensemble models trained with different resolutions to efficiently use distant information for increasing accuracy.

We propose a method that optimally combines coarse features from distant objects and fine features from nearby objects in order to accurately localize a camera. To bridge the gap between OSM methods and satellite methods, we also train our model with both OSM and satellite imagery. The complementary nature of both maps helps generate rich BEVs and neural maps.

1.1 Contributions

The main objective of our thesis is to improve the accuracy of OrienterNet, and to investigate the impact of design decisions used in SNAP. To this end, our contributions are as follows:

- **Bird's Eye View Mapping.** We improve OrienterNet's BEV mapping by replacing its forward mapping strategy with an inverse mapping strategy, resulting in faster inference and more accurate localization.
- **Pose Estimation.** We perform a comprehensive study of two different matching strategies: Exhaustive Matching and RANSAC-like matching.
- **Increasing BEV Depth.** We demonstrate the importance of leveraging distance visual cues for accurate visual localization.
- **Multi-scale Model** After justifying our design decisions, we propose a single strong multi-scale model that learns fine and coarse features and utilizes both near and distant features for localization. This network can be used to localize on much larger maps than Orienternet. We release our model weights, code, and a demo script that allows easy testing of images anywhere in the world where OSM maps are available.
- **Satellite Fusion**. We additionally train a model with both OSM and satellite maps, which further enhances localization accuracy. We demonstrate the benefits of the added information that comes with satellite maps.

1.2 Contents

This thesis is structured as follows:

[Chapter 1] Introduction. In this chapter, we introduced the problem setting and summarized our key contributions.

[Chapter 2] Related Work. This chapter provides relevant context for our work and establishes our motivation. We also provide a comparison between OrienterNet and SNAP that provides better context for some of our design decisions.

[Chapter 3] Method. This chapter introduces our proposed method, which is a single model that learns multi-scale BEV and map representations in order to boost accuracy and scalability.

[Chapter 4] Experiments. In this chapter, we incrementally build our proposed method, with justifications for every design decision. We carefully evaluate the applicability of SNAP's design decisions in our method. Finally, we evaluate a variant that uses satellite imagery to produce further improve localization.

[Chapter 5 Discussion and Conclusion. We summarize the work done in this thesis and reflect on the limitations of our method. We also discuss potential future directions that would be interesting.

Chapter 2

Related Work

Visual localization or camera relocalization is an important and active area of research in computer vision. Given an image (or a sequence of images), the goal is to find the camera's pose relative to a specific representation of a scene. The representation can be one of many forms including an explicit 3D representation like a pointcloud, a 2D representation like a satellite image, or an implicit 3D representation in the form of weights of a neural network.

2.1 Visual Localization

Image Retrieval. Visual Place Recognition (VPR) or image retrieval is the task of estimating the geographical location of the place visible in a given image by searching for its closest match based on appearance among images in a database. Here, the scene is sparsely represented simply in the form of reference images. [1, 45, 7, 37, 2, 12, 40] Often, these methods consist of a reranking stage, which aims at refining the retrieval results usually with geometric verification. These methods have to be invariant to seasonal changes (eg. snow), dynamic objects (eg. pedestrians and cars), and geometric variations such as changes in rotation, translation and scale in the reference image. They also have to efficiently search through potentially large databases of image descriptors to localize in a large area. [5] recently conducted a survey in this field.

These methods require the scene to be densely covered by the images, which restricts their real-world applications. Differently, our work only requires a 2D Map of the region to localize a single query image.

Localization with 3D Maps. To obtain fine and accurate 6-DoF localization, the query image is typically localized against a 3D scene representation such as a 3D pointcloud. These maps are built using Structure-from-Motion (SfM) [43, 11, 25, 38] methods which construct the 3D pointcloud from a set of images. SfM pipelines usually involve several steps including feature detection [6, 8, 50], feature matching [26, 33, 51], and pose refinement [36, 48]. These 3D pointclouds are rich in semantics and geometry but are expensive to store and maintain. Due to the associated costs, it is difficult to use such methods for on-device localization in various robotics applications.

Localization with 2D Maps Localizing on 2D maps is the task of estimating the camera's pose, given a query image and a map tile containing the ground truth pose. This is typically reduced to

a problem of 3-DoF pose estimation as the camera height and gravity direction are often assumed to be easily accessible. The most widely used 2D maps are satellite imagery [9, 10, 23, 41]. A widely used approach is to estimate only a coarse position using tile/patch retrieval [17, 42]. Works like HC-Net [49] aligns a warped ground image with a corresponding GPS-tagged satellite image using homography estimation. In our work, we primarily use OSM maps, but also train a variant of our final model using satellite imagery. We show that both maps can complement each other to generate a rich neural map.

C-BEV [9] is a recent work that learns to estimate 3-DoF poses using satellite imagery without explicit pose supervision. They use contrastive learning with a patch retrieval objective, and score patches by exhaustively matching a BEV. A network learns to produce good BEVs as this helps better score the patches, and consequently leads to a lower patch retrieval loss. However, they are trained on map tiles covering 70m x 70m, and thus their BEV cannot leverage distant visual information for localization. In our work, we propose an approach that learns to use information up to 128m away.

Other works like [4, 15, 29, 14] propose methods that can localize query images/panoramas using indoor floor plans. LaLaLoc, for instance, renders the ground view using the 2D floor plan and optimizes the camera pose using the global representation of the rendered view. F3Loc [4] differently localizes a sequence of images by using a ray-based representation and fusing single and multi-view predictions.

2.2 Bird's-Eye-View

A bird's-eye-view map is a geometrically structured representation that is widely used by autonomous agents for perceiving its surroundings. It comes with several benefits: they are a compact 2D representation of the 3D scene and are similar to the mental map that humans use to understand and navigate their surroundings. They are also closer to the Euclidean space in which autonomous agents act, which make them an attractive choice in many applications.

The main challenge of obtaining a Bird's Eye View from a 2D image is the ambiguity that arises due to the loss of depth information during the projection of physical world onto the image plane.

BEV generation in recent works typically take one of three forms: (a) forward mapping (back-projection) (b) inverse mapping (projection), or (c) attention-based mapping.

Forward Mapping. Prior works in this area [31, 16, 18] typically predict pixel-wise depth estimation along with image features. Given the camera's calibration parameters, the depth estimates can be used to back-project image pixels into 3D. Since depth estimates are often uncertain, the model instead estimates a depth distribution. The image features are then spread out along the ray, weighted by the predicted depth distribution. When depth is perfectly certain, the feature will be at a single point on the ray. OrienterNet, differently, predicts scale instead of depth, to decouple the camera's focal length from the depth estimation. The estimated scale is the ratio of object size in the image plane to the object size in 3D.

Inverse Mapping. Some methods initialize a 3D point grid and "pull" image features towards them by projecting onto the image plane and bilinearly interpolating/subsampling features at the

projected points. SimpleBEV [13] does this without estimating depths, which results in every point along a ray being assigned the same feature. SNAP, instead predicts the pixel-wise depth and interpolates the depth score vector along with the image features. This allows occluded and free-space points to be weighted lower than points belonging to an object's surface. PointBEV [3] adopts this method while reducing the computational and memory costs of the grid by using a sparse grid instead. In our work, we use a monocular variant of the inverse mapping method introduced in SNAP.

Attention-based Mapping. Different from the above approaches, some methods use the more recently popular Transformer architecture [46] in their mapping procedure. [32] uses an attention mechanism mapping vertical columns (scanlines) to polar rays, which is conceptually very similar to the forward mapping implemented in OrienterNet.

Chapter 3

Method

In this chapter, we formulate the problem in Sec. 3.1, provide an overview of our method in Sec. 3.2, and then present our proposed method's three main components, namely Bird's-Eye-View (BEV) Mapping (Sec. 3.3), Neural Map Encoding (Sec. 3.4), and Pose Estimation (Sec. 3.5). In Sec. 3.6, we describe our multi-task training loss. See Fig. 3.1 for an overview of our method.

3.1 Problem Formulation

Given a single RGB image **I** and its calibration parameters, the goal is to estimate the camera's 3-DoF pose (x, y, θ) consisting of position (x, y) and orientation θ, with respect to an East-North-Up (ENU) world coordinate frame.

Inputs: (A) Single RGB image **I**, (B) gravity direction and intrinsics **K**, and (C) a 2D map tile that contains the ground truth camera pose. This can be obtained by using a coarse location estimate such as a GPS measurement or retrieval-based estimate. We propose a method that uses maps from OpenStreetMap (OSM), and one that uses satellite imagery.

For uncalibrated images, the gravity direction and focal length can be inferred using methods such GeoCalib [47], Perspective Fields [20] and [27]. These methods return gravity direction and camera intrinsics.

Since the goal of our method is to improve the accuracy and scalability of OrienterNet, our problem formulation remains the same.

3.2 Method Overview

Our method overview is illustrated in Fig. 3.1. We propose a single network containing two branches that infer features and maps in different scales. Like OrienterNet, our method is trained fully end-to-end through only pose supervision. Intuitively, during training, the gradients push the network to learn a BEV and neural map with sufficient semantic and geometric information in order to reduce the localization loss.

BEV Mapper. We first use a CNN and a simple MLP to predict a multi-scale (fine and coarse) set of features and scales. Two gravity-aligned 3D grids ranging up to 32m and 128m "pulls" features and scale scores, which are then combined using a Fusion MLP. Each grid is then vertically max pooled and fed through a BEVNet to produce refined BEV features and a confidence map.

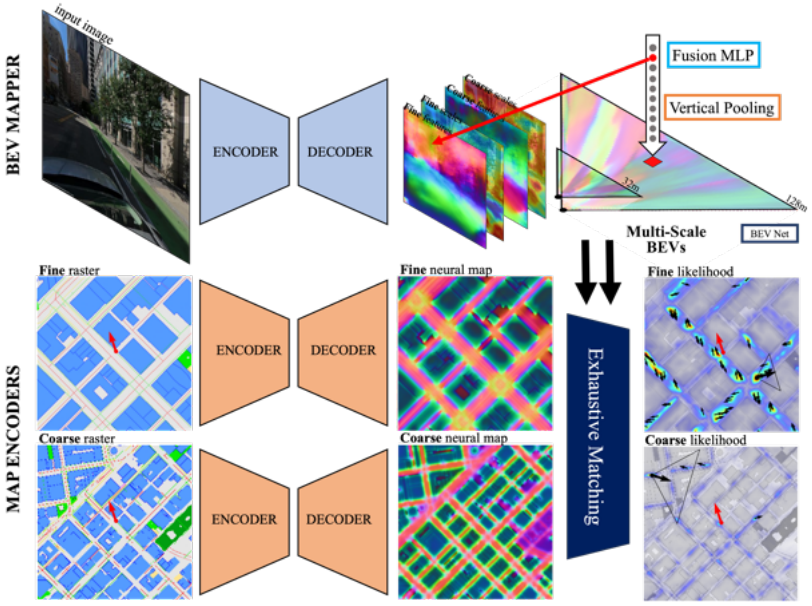

Figure 3.1: **Method Overview**. BEV Mapper: We extract features and scales from the input image, which are processed in two multi-scale gravity-aligned 3D grids and vertically max-pooled. The BEVNet then refines the features and outputs a confidence map per BEV. MAP Encoder: We compute two neural maps at different resolutions. During training, they cover areas of different sizes. Matching: We exhaustively match each BEV with its corresponding neural map to produce two probability volumes. The network is trained end-to-end and learns complementary multi-scale features to effectively disambiguate poses on a large map.

Map Encoder. We use two separate map encoders to produce neural maps at varying resolutions.

Matching. We then exhaustively match both BEVs against their corresponding neural maps. We use an uncertainty-weighted loss to drive the network towards learning an optimal combination of both branches. During inference, we fuse the two scaled probability volumes to get our final pose estimate.

3.3 Neural Bird's-Eye-View

In this section, we introduce the mapping strategy we use to obtain two BEV representations at different resolutions from a single image.

3.3.1 Image to BEV Mapping

Image Features and Scales. For a query image $\mathbf{I}$, our convolutional feature extractor Φ_I outputs a feature image $\mathbf{F} \in \mathbb{R}^{H \times W \times 2C}$, from which we extract two sets of C-dimensional features: $C_{\textit{fine}}$ and $C_{\textit{coarse}}$. For each set of features, a simple MLP is used to predict a scale score vector $\mathbf{S}$ per pixel (u, v), where each element corresponds to a scale in a set of log-distributed scales, as in OrienterNet. A separate scale classifier for each feature type allows specialization towards a single input type. Learning the scale instead of directly estimating depth allows us to decouple the focal length f from the depth d and assumes that f is an input to the method. The scale s of a pixel is related to its depth through $s = \frac{f}{d}$. Splitting features and scales in this way allows the finer feature set to focus on details like building corners, poles, trees and benches, while the coarser features can focus more on understanding distant visual cues for accurate localization.

The next stages of this method are identical for the coarse and fine branches and are run sequentially. For simplicity, we describe only one of them below.

3D Grid. Inspired by the multi-view fusion introduced in SNAP, we build a 3D grid consisting of points $\mathbf{P}_i = (x_i, y_i, z_i)$ in the gravity-aligned camera frame *gcam*. We project each point using a perspective projection function $\Pi : \mathbb{R}^3 \rightarrow \mathbb{R}^2$, which consists of a rotation matrix ${}_{\textit{cam}}\mathbf{R}_{\textit{gcam}} \in SE(3)$ and the camera intrinsics which include focal length and lens distortions. ${}_{\textit{cam}}\mathbf{R}_{\textit{gcam}}$ transforms a point in *gcam* to the coordinate frame of *cam*. This gives us a 2D point correspondence, $\mathbf{p}_i = \Pi(\mathbf{P}_i)$ for each 3D point. At these points, we bilinearly interpolate features and scale scores: $\mathbf{F}_i = \mathbf{F}[\mathbf{p}_i]$ and $\mathbf{S}_i = \mathbf{S}[\mathbf{p}_i]$, where $[.]$ denotes interpolation. Note that here we interpolate only from the feature set and scale scores corresponding to this branch. In other words, the fine 3D grid only projects onto the fine features and scales. We then extract a scalar scale score from the interpolated scale score vector: $S_i = \mathbf{S}_i[\frac{f}{d_i}]$, where d_i is the depth of $\mathbf{P}_i$ in the camera frame.

When estimating the scale of a pixel is ambiguous due to the inherent challenges of monocular depth estimation, or challenging due to lighting or blur, the score vector can be assigned a uniform score across all its scales. In this case, the features will be spread across all 3D points along the ray intersecting this pixel and the camera center. On the other hand, when the model is highly confident of the scale of a pixel, the corresponding 3D point is assigned a high score, consequently

pushing down the importance of every other point along that ray. When a 3D point is occluded or in free space, it receives a low scale score.

Fusion MLP. Our Fusion MLP fuses the feature $\mathbf{F}_i$ and the score S_i to produce a new feature $\mathbf{X}_i \in \mathbb{R}^C$. In SNAP's variant of multi-view fusion, the feature prediction and depth estimation benefit from multi-view constraints, which we do not have.

Vertical Pooling. Each column of the 3D grid with fused features are then vertically max pooled to obtain a single cell feature $\mathbf{B}^{pool}_{ij} \in \mathbb{R}^C$, where $\mathbf{B}^{pool}$ is the obtained 2D neural BEV. In doing so, we encourage higher norms for important features in a vertical grid column.

BEVNet Finally, we pass our BEV through a small CNN Φ_{BEV} which outputs our final neural BEV $\mathbf{B} \in \mathbb{R}^m$, where m is the matching dimension, for which we choose a much lower dimension than the latent dimension C to reduce memory consumption during matching. We also output a confidence mask which is applied to $\mathbf{B}$ before matching. This helps avoid uncertain feature predictions in our matching procedure.

We show through our experiments in Sec 4.2 that this method is faster than OrienterNet's forward mapping approach and also results in better accuracies. However, we show that it requires more memory.

BEV Depth and Resolutions. The two BEV grids described above are designed to have the same number of points, but different ground resolutions. This allows us to obtain two BEVs having same number of pixels, but mapping different maximum depth ranges. In our study in Sec. 4.2.2, we demonstrate that utilizing distant visual information is highly beneficial for disambiguating poses, while using high resolution nearby information is crucial for retaining fine accuracies.

3.4 Neural Map

Our method uses two separate map encoders which encode the input map rasters into two neural maps: $\mathbf{M}_{\text{fine}}, \mathbf{M}_{\text{coarse}} \in \mathbb{R}^{W \times H \times m}$,, where m is the matching dimension, similar to the output of the BEVNet in the previous section (3.3). While both encoders are identical, we find that when the input map rasters are too coarse, there is a significant amount of information lost in the pre-processing. To alleviate this issue, we provide a $2\times$ finer map raster for the coarser branch, and extract our desired neural map from the penultimate feature map of its U-Net-style decoder.

Fusing Satellite Imagery. We argue that OpenStreetMap maps and Satellite imagery are complementary in nature. While the OSM maps have clear boundaries and many types of elements, they lack textures, colours, road/curb edges and markings, and other elements that are not in the OSM class set. From visual inspection, we also notice OSM maps missing trees, poles and benches in many instances. This is not unexpected as the source of this data is crowd-sourced and quality of the maps rely on individual contributions. In order to train a model with both maps, we create a dataset of satellite imagery and query map tiles as we do with OrienterNet. Then, we feed the satellite maps to a 1x1 Convolutional layer, and concatenate the output with the OSM map embeddings, before passing the concatenated tensor through our map encoder. We also considered a late fusion method. When trained with dropout as done in SNAP, one could choose either map modality during inference depending on availability. Moreover, the specialized layers per modality will likely learn richer representations. However, this would require much more memory as we would have 4 separate map encoders. Therefore, we do not experiment with this approach.

3.5 Multi-Scale Pose Estimation

To estimate poses, we exhaustively match a BEV **B** against its corresponding neural map **M** to obtain a score volume **V**. We then convert this volume to a probability distribution over camera poses using the softmax operation. These poses are positioned at each map cell and oriented at a fixed number of rotations. For inference, we select the pose with the maximum likelihood in the probability volume. Our method does not use a map prior like OrienterNet does.

Each branch sequentially performs matching and yields two probability volumes at different resolutions. We use the same Fourier-based formulation used by OrienterNet. In our study in Sec. 4.2.4, we compare OrienterNet's exhaustive matcher against SNAP's RANSAC matcher. We find that the exhaustive matcher, despite scoring millions more poses, is far more efficient than the RANSAC-based method.

Chaining Probability Volumes. Sharing information from both branches to localize a query image can be very helpful when the coarse and fine features have a high variance and are complementary. For example, when the coarse BEV's localization assigns equal probabilities to two ambiguous poses, the fine BEV could potentially break the tie using finer-grained information like trees and poles that are visible in the image.

To combine information, we compute the joint probability volumes of both branches. We define this as:

$$\mathbf{P}(\xi_{\text{chain}}|\mathbf{I}, \text{map}_{\text{fine}}, \text{map}_{\text{coarse}}) = \mathbf{P}(\xi_{\text{fine}}|\mathbf{I}, \text{map}_{\text{fine}}) \times \mathbf{P}_{\text{upsampled}}(\xi_{\text{coarse}}|\mathbf{I}, \text{map}_{\text{coarse}}) \tag{3.1}$$

where ξ denotes the camera poses and

$$\mathbf{P}_{\text{upsampled}}(\xi_{\text{coarse}}|\mathbf{I}, \text{map}_{\text{coarse}}) = \text{softmax}(\mathcal{U}(\mathbf{V}_{\text{coarse}})) \tag{3.2}$$

A bilinear upsampling operator $\mathcal{U}$ is used to upsample the coarse score volume $\mathbf{V}_{\text{coarse}}$. We found this method to be very effective when combining single-branch models, which then motivated the use of this method.

3.6 Multi-Scale Training Loss

OrienterNet is supervised by maximizing the log-likelihood of the ground truth pose ξ. The loss is defined as:

$$Loss = -\log \text{softmax}(\mathbf{V}) \tag{3.3}$$

where **V** is the matching volume as defined previously.

In our experiments, we first trained our method using a simple summation of the losses of both branches. This gives us:

$$Loss = -\log \text{softmax}(\mathbf{V}_{\text{fine}}) - \log \text{softmax}(\mathbf{V}_{\text{coarse}}) \tag{3.4}$$

However, we found that this loss does not allow the branches to learn complementary features. During evaluation, we find that multiplying the probability volumes of both branches (which we often call "chaining") does not improve the accuracy of a single branch evaluated in isolation. We hypothesize that the main reason for this is that the two losses are on different scales due to the

varying difficulties of the tasks of both branches. Having both losses weighted correctly is required to ensure that sufficient importance is assigned to both without letting either dominate the learning procedure in an undesirable way.

Recall that during training, both branches encode maps covering different areas to keep the memory requirements equal (their pixel resolutions are equal). The larger map corresponding to the coarse branch contains many more self-similarities than the smaller map. Intuitively, when localizing an image captured at, say, a traffic intersection, increasing the map search area will likely include other similar intersections, thus making the task harder. At the same time, the larger BEV depth of this coarse branch gives it an advantage due to the added information from further distances. These differences between the two tasks make it challenging to know the ideal weights to balance both losses.

Similar summation losses in multi-task settings were commonly used previously with fine-tuned weights [39, 44] for dense prediction and scene understanding tasks. However, [21] shows that model performance is extremely sensitive to weight selection, and they propose a convenient method which can learn the optimal weights between multiple tasks. Their approach suits our training well, as it uses the inherent task uncertainty in order to balance both losses. To this end, we have adopted their method for designing our loss. Our modified loss is as follows:

$$\mathit{Loss} = -\log \text{softmax}\left(\frac{1}{\sigma_{\text{fine}}^2} \cdot \text{scores}\right) - \log \text{softmax}\left(\frac{1}{\sigma_{\text{coarse}}^2} \cdot \text{scores}\right) + \log \sigma_{\text{fine}} + \log \sigma_{\text{coarse}} \tag{3.5}$$

In practice, however, we train the network to predict the log variance $\tau = \log \sigma^2$ for each branch. [21] proposes this alternative as well, as it avoids any division by zero and is thus numerically more stable. Moreover, it constrains the network to learn variances that are valid, unlike Eq. 3.5. This results in the following adapted formula:

$$\mathit{Loss} = -\log \text{softmax}\left(e^{-\tau_{\text{fine}}^2} \cdot \mathbf{V}_{\text{fine}}\right) - \log \text{softmax}\left(e^{-\tau_{\text{coarse}}^2} \cdot \mathbf{V}_{\text{coarse}}\right) + \frac{\tau_{\text{fine}}}{2} + \frac{\tau_{\text{coarse}}}{2} \tag{3.6}$$

In our experiments in Sec. 4.3, we show this brings substantial improvements over the simple loss (Eq. 3.4)

Chapter 4

Experiments and Evaluation

In this chapter, we justify our design decisions, evaluate our proposed method and compare it with OrienterNet [34]. Table 4.4 shows that our method is capable of achieving over 60% Recall under 20m on maps as large as 512m x 512m, which is a 20% higher score than OrienterNet.

In Sec. 4.1, we describe our experimental setup, which includes training and validation datasets, evaluation metrics, backbone architectures, and training details. In Sec. 4.2 and Sec. 4.3, we incrementally justify design choices that can be applied to single-scale and multi-scale architectures. We closely analyse key components of the network, such as BEV Mapping (Sec. 4.2.1) and Matching (Sec. 4.2.4) for single-scale and multi-scale models. In Sec. 4.4, we compare our method and its variants against OrienterNet. We conclude this chapter with visualizations that illustrate the strengths and weaknesses of our proposed method.

4.1 Experimental Setup

Datasets. We train and evaluate our model on data extracted from Mapillary and OpenStreetMap (OSM), similar to OrienterNet [34]. Images and Poses: We extract a dataset from Mapillary consisting of 813,531 training and 1,911 validation images. The images are taken in 12 cities (same as OrienterNet) in Europe and the USA. Training and validation splits cover disjoint areas in the same cities. Mapillary also returns the 6-DoF pose (x,y,z,roll,pitch,yaw) (obtained using a fusion of SfM and GPS), camera calibration, and a GPS measurement. The roll and pitch are used to position the gravity-aligned 3D grid in the camera frame, and the camera calibration is used to project the grid points onto the image plane. We do not use the height z, and use the GPS measurement only for evaluation purposes. The x,y, and orientation θ serve as the Ground Truth pose that we train and evaluate our method with. 2D Maps: We query OSM for each city and create tiles at different resolutions for our various experiments. The resolutions we use are 0.5, 1, 2, and 4 meters per pixel (mpp). These tiles consist of 3 channels representing areas, lines, and nodes.

During training, we always use a map with radius 256 pixels. Depending on the chosen resolution, this could result in maps with radii ranging from 64m to 512m. Restricting the size of the map in pixels ensures the cost of matching is always the same during training. During evaluation, we always use a map with radius 256 meters; this allows fairly comparing methods over the same metric search space, regardless of pixel resolution. For our experiments using satellite imagery (Sec. 4.2.3), we query Bing Maps [28] and create RGB tiles at the same ground resolutions as the

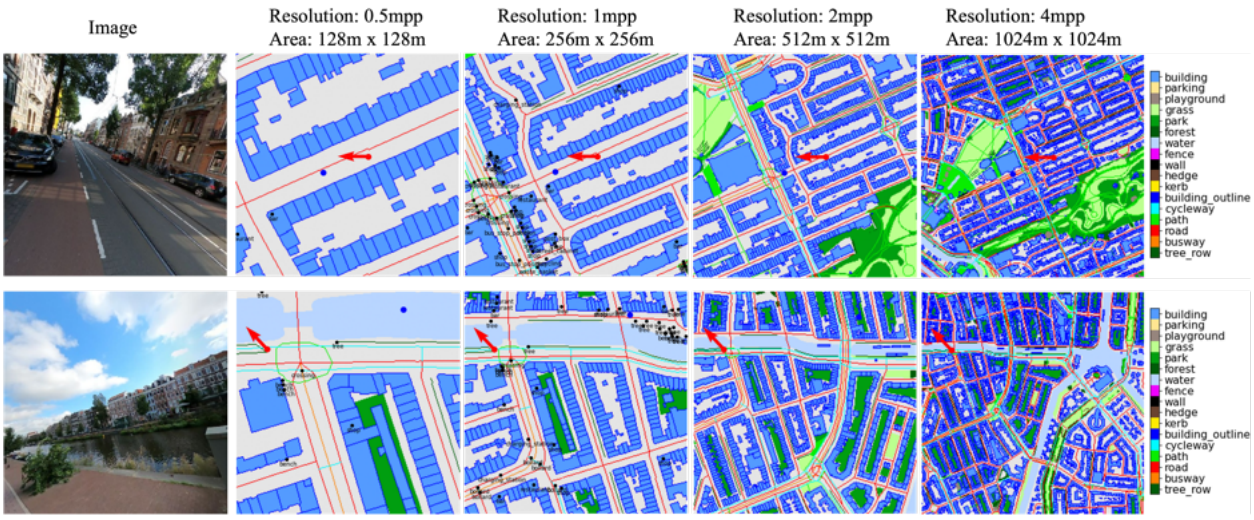

Figure 4.1: **Samples of images and multiscale maps from our training dataset**. 3-DoF pose (x, y, θ) is shown with a red arrow. During training, we use maps with radius 256px, which results in coarser maps covering larger areas. As the map areas grow larger, localizing a single image becomes increasingly more challenging due to the higher number of ambiguous poses. Note how it is much easier for us to localize the images above on the smallest map.

OSM tiles.

Evaluation Metrics. We report recall of positional and orientational errors at six thresholds: 0.5/1/2/5/10/20 m and 0.5/1/2/5/10/20 degrees. We also report mean errors and uncertainty. We measure uncertainty as the exhaustive entropy of the matching score volume, normalized over number of poses. The normalization allows us to directly compare uncertainties computed over maps of varying areas and resolutions. Intuitively, the uncertainty of localization will be high when there are multiple ambiguous poses on the map that agree with our BEV.

We run our evaluation in a Retrieval and a Precision paradigm, with a search radius of 256m and 10m respectively. This allows us to fairly evaluate methods that fail to differentiate between ambiguous poses on a large map but are capable of obtaining sub-meter accuracies in a smaller search space. For limiting the search space, we do not use a small map, but we mask out poses outside the search space in the matching score volume. For evaluation, the ground truth poses are always within a 128m radius (for search radius 256m) or 10m radius (for search radius 10m) of the map.

Backbone Architectures. As described in our method's illustration in Fig. 3.1, our proposed method contains a BEVMapper and a Map Encoder. For our BEVMapper, we extract image features and scale scores using a ResNet-101 backbone and an FPN decoder, and we encode the map raster(s) using a VGG-19 backbone encoder and a U-Net decoder. For all our design experiments (Sections 4.2 and 4.3), we opt for a ResNet-18 backbone for the BEVMapper for faster iteration.

Training Details. We train all our models using a constant learning rate of 5^{-10} with the ADAM optimizer. All our design experiments are trained with a batch size of 4 for 320k iterations. This corresponds to 1.5 epochs. While this is insufficient for convergence, these models still provide meaningful insights into the models' behaviour and require much less computation. With our best architectures, we then fully train 2 of our models until convergence with a batch size of

12 split across 4 GPUS having 24 GB VRAM each. Our OSM-only model stops improving after 800k iterations (11.8 epochs), and our OSM+Satellite model stops improving after 940k iterations (13.9 epochs). Fully training these models takes 3-4 days on an RTX 4090 GPU.

Like in OrienterNet, our image and maps are 128-dimensional, but are projected to an 8-dim space for matching. Unlike OrienterNet, we do not learn map priors. We use 33 scale bins for our scale estimation. During training, we always use a map mask that limits the search radius to $0.75\times$ the map radius, and our map radius is always $2\times$ the BEV depth range. For our evaluations, we use 64 number of rotations during matching.

4.2 Design Decisions: Single-Scale Models

In this section, we report our experiments that justify our design decisions on single-scale models. These choices are then used in the individual branches of our fully trained multi-scale model. In the next Section 4.3, we discuss improvements that are specific to the multi-scale architecture.

4.2.1 Neural BEV: Forward vs Inverse Mapping

We study the performance of two image-to-BEV mapping methods - forward and inverse mapping. We compare the forward mapping strategy as used in OrienterNet, and we adapt the inverse mapping strategy introduced in the Multi-View Fusion component in SNAP.

To evaluate these methods, we benchmark the speed and memory required for a forward pass through each method with a batch size of 2 on an RTX 3090 GPU. Results are shown in Tables 4.1 and 4.2. We ignore the costs associated with rectifying the image (a requirement only for the forward mapping) as this is part of the dataloading and not part of the forward pass. We also train a model with each mapping method and report localization results in Table 4.3.

We learn that the inverse mapping strategy benefits from a 2.5x speedup over the forward mapping while requiring nearly twice as much memory. The inverse mapping's most expensive component is the Fusion MLP, which processes each point of the 3D grid containing 200k points (64 x 129 x 24).

The localization results of both trained models show that while the positional recalls are on par, the inverse mapping outperforms the forward mapping method in orientational recalls. This is likely either due to the forward mapping's Cartesian Projection component spreading features along multiple bearing angles, or the Fusion MLP producing higher quality features.

In Fig. 4.2, we visualize the learned BEVs and their confidences and compare them against a crop of the learned neural map at the ground truth pose. The inverse mapping's BEV appears to be more geometrically accurate - it retains the rounded building wall and straight roads.

We decide to adopt the inverse mapping strategy for all our subsequent experiments due to the speedup and better learning, despite the higher memory requirement.

4.2.2 Neural BEV: Increasing Depth

We now show the localization accuracies of models trained at increasing BEV depth ranges, namely 32m, 64m, 128m. Note that we trade-off resolution for increased depth ranges, and thus

Component	Time (ms)
Polar Projection	19.67
Cartesian Projection	0.11
Total Time	19.78 ms
Peak GPU Memory	1.26 GB

Table 4.1: Forward Mapping Cost.

Component	Time (ms)
Build 3D Grid	0.326
Project Grid	0.25
Interpolate Features	2.92
Interpolate Scale Scores	0.23
Fusion MLP	3.8
Vertical Pooling	0.29
Total Time	7.816 ms
Peak GPU Memory	2.49 GB

Table 4.2: Inverse Mapping Cost.

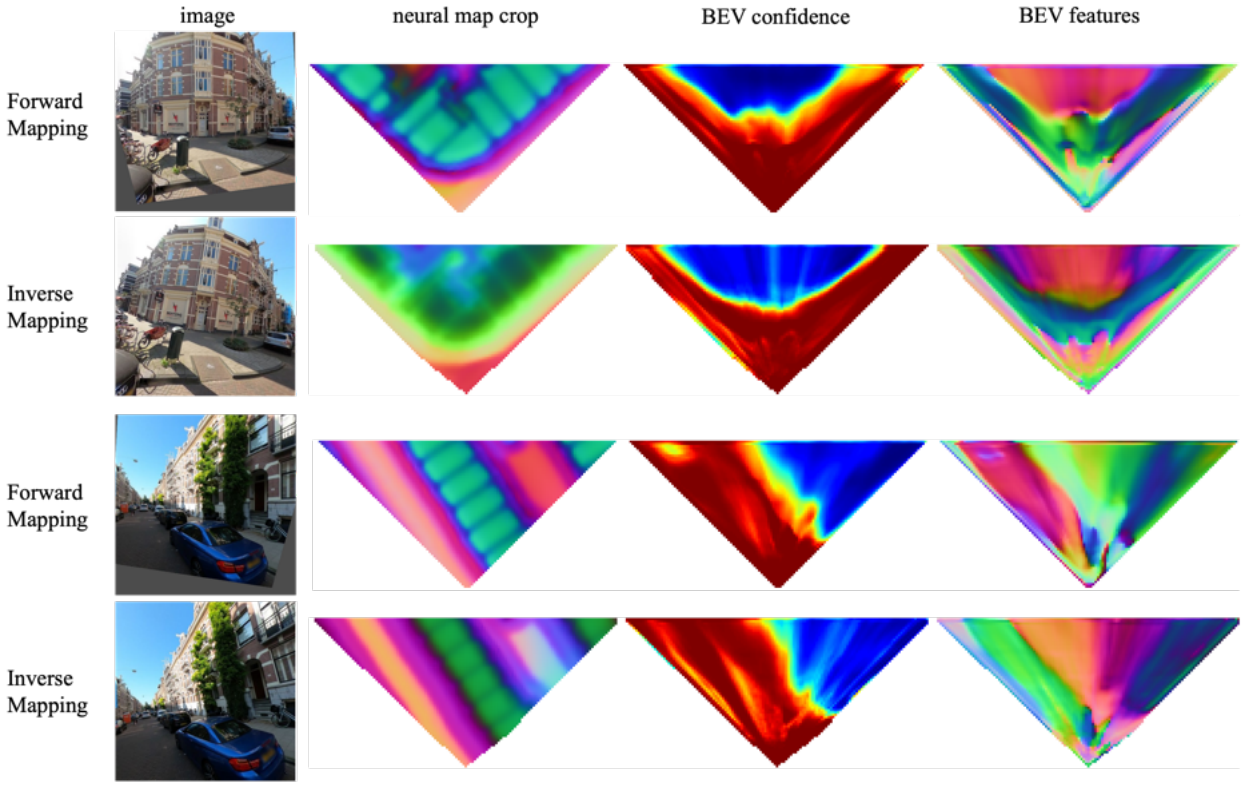

Figure 4.2: **Forward Mapping vs Inverse Mapping:** Visualizing learned Neural BEV and confidence. In the first two rows, the curved building wall appears to be better represented by the inverse mapping's BEV and its confidence map (blue denotes low confidence). In the second example, the BEVs are similar, with the inverse BEV showing slightly straighter roads. Notice the inverse mapping does not require rectified images, and can thus occasionally benefit from few additional visual cues that are otherwise discarded during rectification.

experiment	uncertainty	xy mean error	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	yaw mean error	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
Forward Mapping	0.36	**143.61**	**1.36**	**4.45**	**12.09**	19.73	23.08	25.8	70.16	**4.29**	8.53	16.17	29.77	37.52	41.86
Inverse Mapping	0.36	144.07	1.2	4.19	11.72	**19.99**	**23.29**	**25.95**	**64.49**	4.19	**9**	**16.9**	**31.92**	**39.19**	**43.59**

Table 4.3: Forward Mapping vs Inverse Mapping Localization Results.

these models' computational costs are identical. The resolutions are 0.5mpp, 1mpp, and 2mpp respectively.

We also study the accuracies obtained when chaining pairs of models. As described in Sec 4.2.3, chaining is done by multiplying different probability volumes.

The Single-Scale section of Tables 4.6 and 4.2.4 show that when we increase depth, we achieve large gains in localizing in a large search space, but at the cost of losing finer accuracy in smaller search spaces. The larger BEVs are very effective at disambiguating poses, which is reflected by their decreasing uncertainties. This is because visual cues present further away are highly informative and help rule out poses that have similar nearby objects. Even the presence or absence of occlusions (for example, building vs no building) is very useful information. Chaining them retains the best of both worlds. By fusing their probability volumes, we essentially share information between both models and choose the pose estimate which wins the consensus of all models involved.

We saw that simultaneously reducing resolution and increasing depth range is beneficial. In order to understand the effect of reducing resolution alone, we evaluate 3 of our models with all BEVs ranging only up to 32m. To achieve this, we simply mask out pixels in the BEV that are further than 32m from the camera, and evaluate as usual.

Model	XY Mean Error (m)	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	Yaw Mean Error (°)	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
32m	144.07	1.2	4.19	11.72	19.99	23.29	25.95	64.49	4.19	9	16.9	31.92	39.19	43.59
64m	147.13	0.89	3.3	10.62	19.83	22.66	25.54	66.28	3.51	7.17	13.29	27.21	37.31	42.49
128m	152.42	0.52	1.47	4.97	14.7	18.16	20.51	73	2.2	3.92	6.75	17.22	28.78	36.05

Table 4.4: Equal Depth of 32m, Coarser Resolution. We use only the first 32m of the BEV for localization of all 3 models. Resolutions are 0.5mpp, 1mpp, and 2mpp for the 32m, 64m and 128m models respectively.

4.2.3 Neural Map Encoding

Our map encoder takes as input a 3-channel map raster containing the ground truth pose, and outputs an 8-dimensional neural map which is used for matching. When generating map rasters at coarse resolutions, there is a loss of information that may cause positional inaccuracies. Instead of allowing this information loss to be a function of the data pre-processing, we instead choose to input map rasters at twice the resolution of the desired output neural map. If the desired neural map resolution is 2mpp, then our input is 1mpp. Since we always output neural maps with shape 256px x 256px, this implies that we now input a map raster with shape 512px x 512px. In doing so, we allow the network to carefully choose how to drop information in a way that does not hurt localization accuracy.

Our map encoder uses a VGG backbone with a simple U-Net style decoder. Our final neural map is produced by the penultimate layer of this decoder.

At the bottom of the Single-Scale section of Tables 4.6 and 4.2.4, we see that this method consistently yields improvements over its coarser counterparts. Note that the matching is still done at the same resolution as before, and the only cause for improvement here is that the input to the map encoder is at a higher resolution.

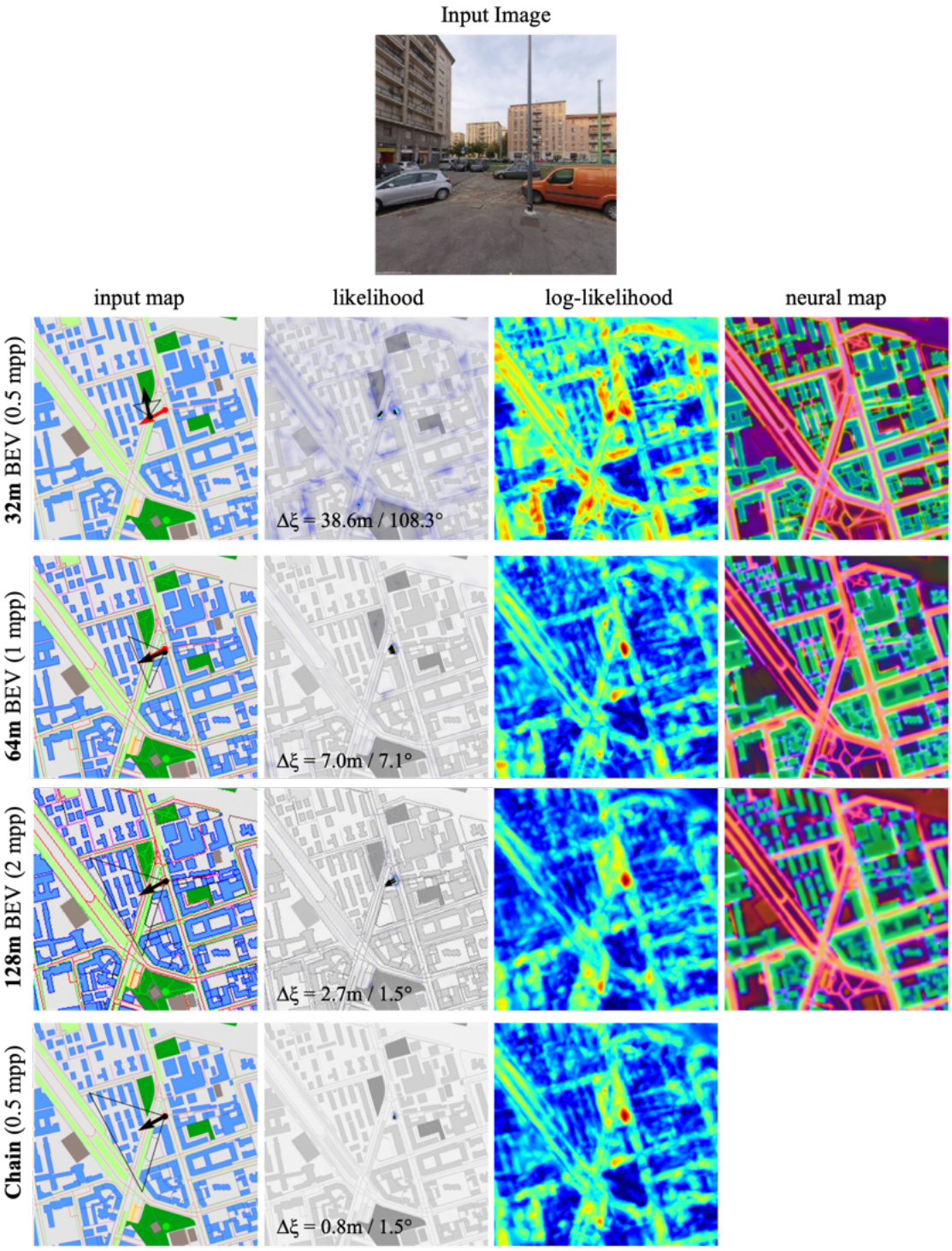

Figure 4.3: **Increasing BEV Depth Range yields large improvements**. Chaining results of multiple models trained at different depths and resolutions drastically reduces uncertainty (see sharper likelihood) and results in high accuracy. GT pose (red arrow), estimated pose (black arrow), and the BEV's outline is overlayed on the input map. In this example, the buildings that are far away are crucial for accurate localization.

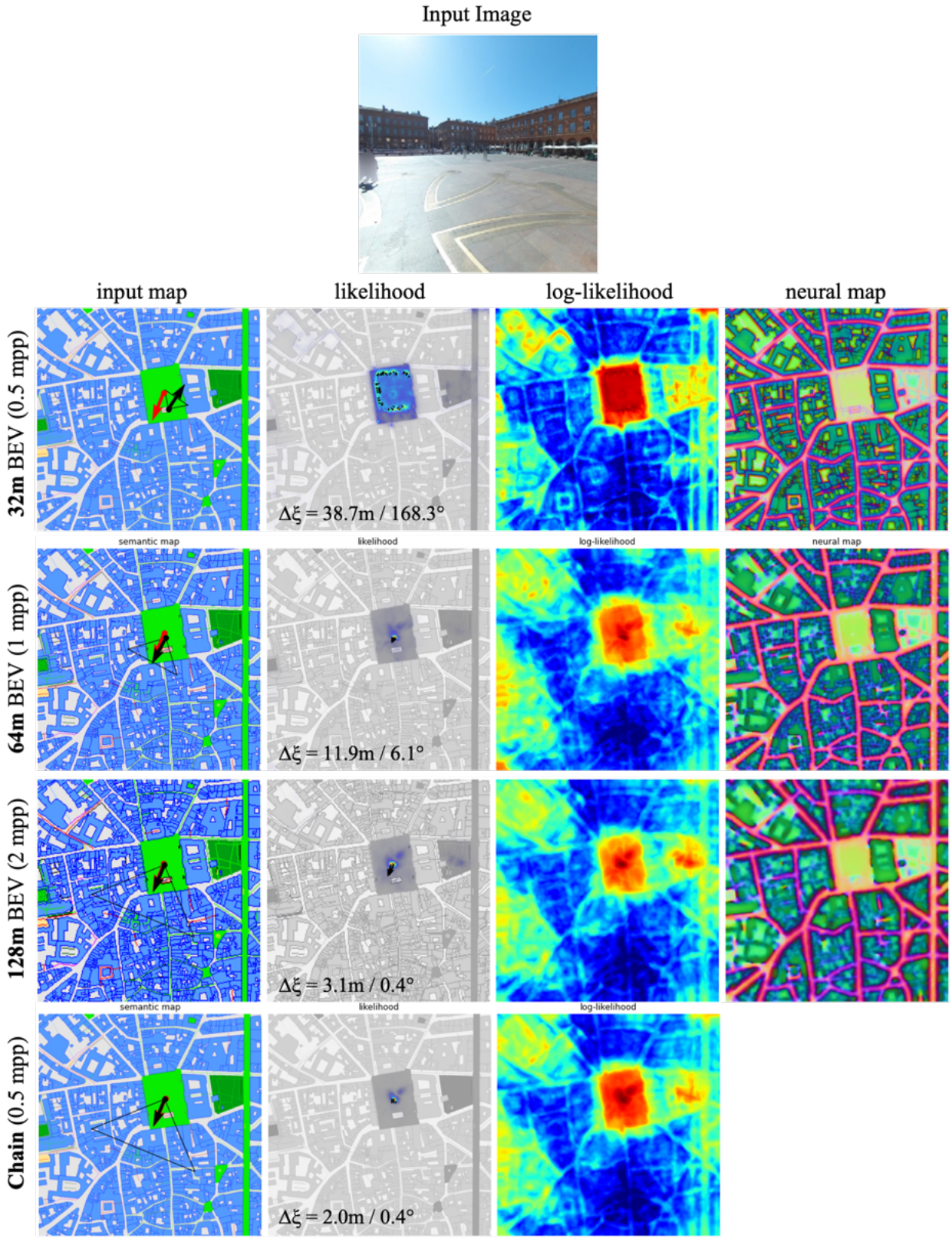

Figure 4.4: Similar to 4.3. In this example, localizing with the 32m BEV results in high uncertainty as the visible buildings do not fall within its range. Increasing the range narrows down possible poses.

4.2.4 Pose Estimation: Exhaustive vs RANSAC

In this section, we thoroughly evaluate the two pose estimation methods: exhaustive (as in OrienterNet) and RANSAC (as in SNAP). Since matching the neural BEV against the neural map is a key component of the pipeline, it is crucial that our method is capable of accurately and efficiently finding the best match. We are also interested in learning their scalability with respect to the neural map size.

To fairly compare these methods with each other, our setup is as follows. We evaluate OrienterNet's pretrained model without its BEV mapper. Instead, we extract a BEV cutout at the ground truth pose and match this against the predicted neural map (see Fig. 4.5). This acts as a perfect BEV and a good matcher should accurately estimate its position. We evaluate both matchers on our full validation dataset and report results in Table 4.5. We configure the exhaustive matcher to use 64 rotations, and the RANSAC matcher to sample 10,000 poses, as used during training in OrienterNet and SNAP respectively.

We see that despite exhaustively matching millions of poses, the exhaustive matcher is significantly more efficient than the RANSAC method, which scores only 10k poses. Scoring many more poses leads to a much higher accuracy. The main reason for the large speedup is the use of Fourier-based convolutions. Here, the rotated BEVs are transformed to the Fourier domain, followed by a single element-wise multiplication, and then applying an inverse FFT operation to obtain the result. This has a computational complexity of $O(M \log M + N \log N + M)$ and is especially advantageous when the kernel size is large, like in our case. On the other hand, the computational complexity of the RANSAC based matching is $O(MN)$. This is dominated by the similarity computation, where each cell in the BEV is matched against each cell in the neural map. When matching with a neural map with radius 256px, $M = 256 \times 256 = 65,536$ and $N = 64 \times 129 = 8,256$, which gives us $M \log M + N \log N + M = 866,811.8$ and $MN = 541,065,216$

Our RANSAC implementation is written in PyTorch and is nearly identical to SNAP's JAX implementation. We do not use SNAP's grid refinement for this evaluation, since that is only used during evaluation and is an additional step to the RANSAC matching.

In Fig. 4.6 and 4.7, we plot the matchers' Position and Orientational Recall, Time, and Peak GPU memory (for a single forward pass), for varying map sizes and number of rotations/sampled poses. We test on maps with radii 256, 320, 360, and 448. We also study how these metrics change with number of rotations (for the exhaustive matcher), and number of sampled poses (for the RANSAC matcher).

The Exhaustive Matcher yields very high accuracy even at 64 rotations. It also scales well to larger map sizes. Matching on a 448px radius map at 64 rotations takes 37.0ms with a peak GPU memory of 2.88GB, and achieves a 100% Recall under 5m.

On the other hand, the RANSAC matcher scales poorly with respect to all metrics - accuracy, time and memory. Matching on a 448px radius map with 10k sampled poses takes 121.5/18.8 and only achieves a 65.69% Recall under 5m. This shows that larger map sizes necessitate a larger set of sampled poses, which are very expensive. The most expensive components of the RANSAC matching are the Pose Scoring and similarity computation, despite an efficient batched implementation.

From this study, we learn that even though the RANSAC matching seems like a promising alternative to exhaustive matching due to its sparser matching, it is significantly more expensive. We thus decide to stick with OrienterNet's exhaustive matching for our proposed model.

Matcher	# Scored Poses	R @ 1m	R @ 3m	R @ 5m	R @ 1°	R @ 3°	R @ 5°	Peak Mem. (GB)	Total Time (ms)
Exhaustive	4,194,304	75.49	100	100	31.86	96.08	98.53	1.45	22
RANSAC	10,000	10.78	66.67	93.14	14.22	26.47	60.78	6.85	71.2

Table 4.5: Exhaustive vs RANSAC matching. We synthesize a perfect BEV by cutting it out of Orienternet's neural map at the GT pose, and evaluate it on our validation dataset. Here, the neural map's radius is 256px. The exhaustive matcher is significantly more accurate and efficient. It scores millions more poses in a fraction of the time taken by the RANSAC matcher, thanks to the highly efficient FFT Convolution.

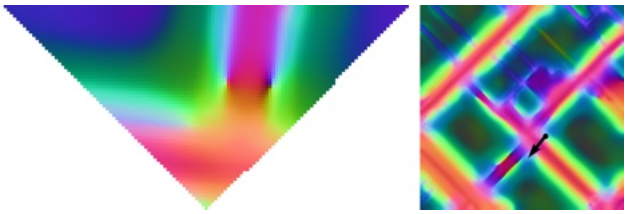

Figure 4.5: Example of Query BEV (left) cropped from Neural Map (right) at GT Pose (black arrow). In Section 4.2.4, we evaluate and compare Exhaustive vs RANSAC pose estimation using this BEV.

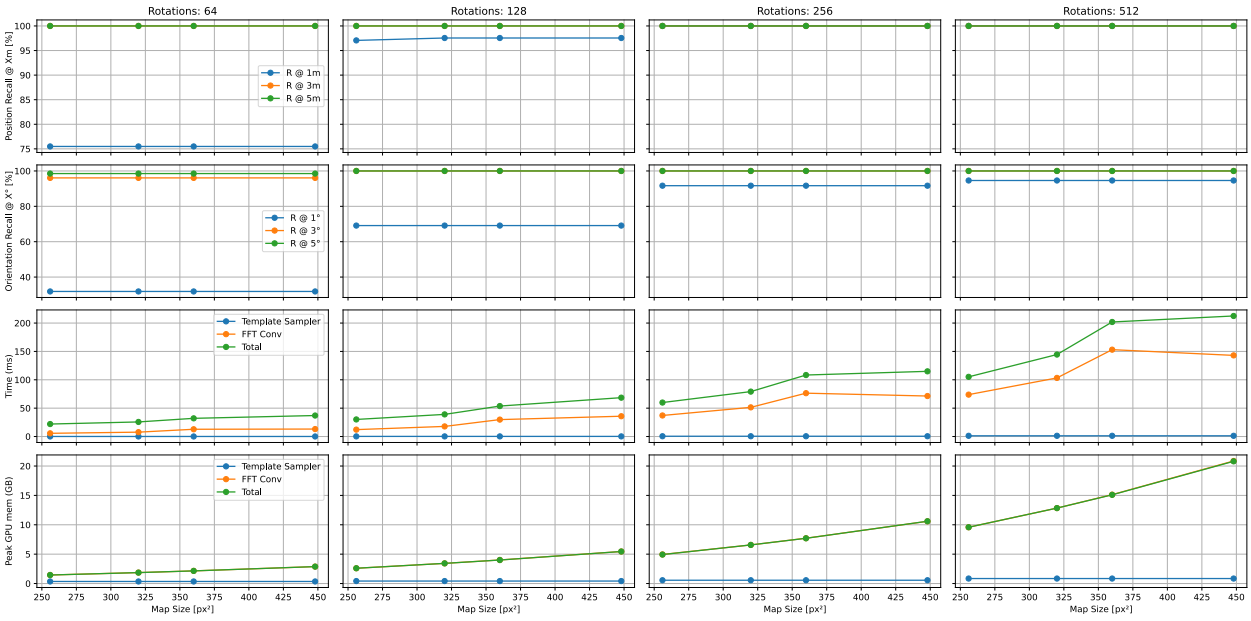

Figure 4.6: **Exhaustive Matcher**: Measuring accuracy, time, and memory on varying map sizes and number of rotations

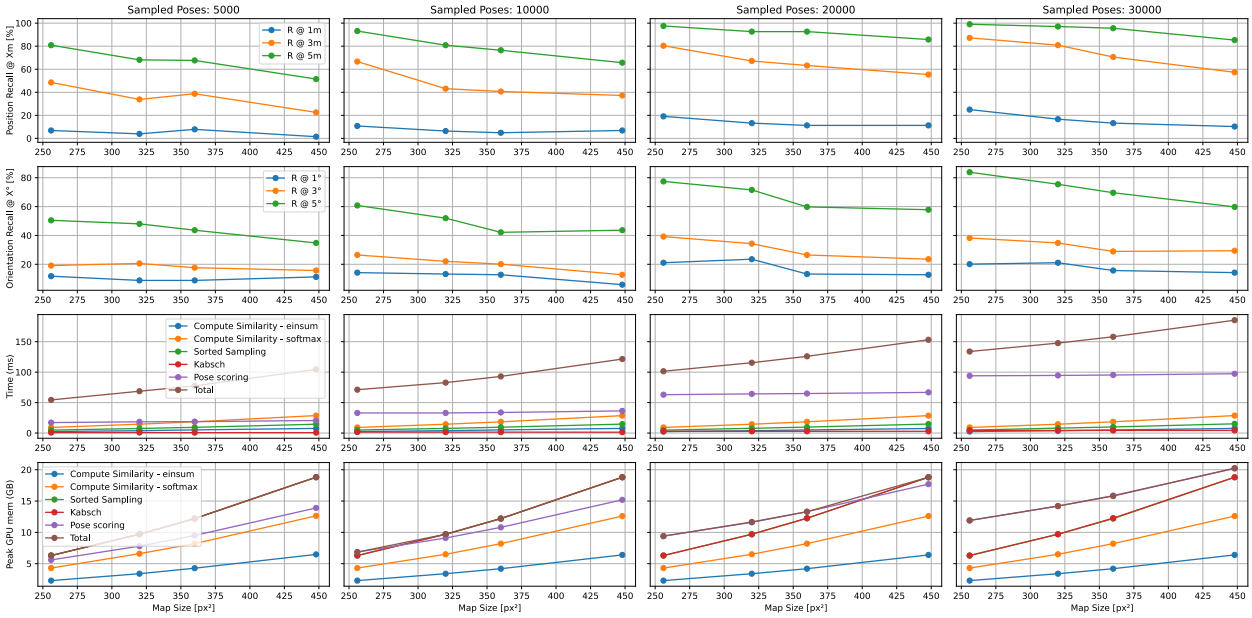

Figure 4.7: **RANSAC Matcher**: Measuring accuracy, time, and memory on varying map sizes and number of sampled poses

Model Type	idx	Model	Uncertainty	XY Mean Error	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	Yaw Mean Error	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
1. Single-Scale	a	32m	0.36	144.07	1.2	4.19	11.72	19.99	23.29	25.95	64.49	4.19	9	16.9	31.92	39.19	43.59
	b	64m	0.27	121.68	1.31	5.23	16.17	31.08	36.05	38.51	57.86	5.91	11.46	22.29	40.29	47.15	50.71
	c	128m	0.21	113.77	1.2	3.61	12.14	31.61	39.25	42.8	53.25	6.44	12.3	24.18	43.85	51.23	55.15
	d	Chain(32m, 64m)	0.24	112.61	2.2	6.33	19.52	34.33	39.25	42.39	53.61	6.8	13.13	25.07	45.21	51.75	55.1
	e	Chain(64m,128m)	0.18	102.75	1.62	5.97	17.69	37.73	44.06	47.62	48.36	7.95	14.76	28.57	49.87	56.78	59.5
	f	Chain(32m,128m)	0.25	103.48	1.52	6.07	19.05	35.85	42.44	45.94	49.01	**8.16**	**15.12**	28.68	49.29	55.99	59.55
	g	Chain(32m, 64m, 128m)	0.2	**97.16**	**2.3**	**7.22**	**20.72**	**38.83**	**45.21**	**48.72**	**47.51**	7.95	14.81	**29.67**	**51.7**	**58.45**	**61.85**
	h	64m - Finer Map Input	0.25	118.93	1.2	5.55	16.17	29.09	35.37	38.51	57	5.86	11.25	22.19	40.71	48.67	53.48
	g	128m - Finer Map Input	0.19	111.89	1.36	4.45	14.13	32.55	40.61	44.53	52.97	6.96	13.5	25.75	45.63	52.8	55.99
2. Multi-Scale	a	img: sep, map: shared [32m]	0.34	132.95	1.26	4.87	12.72	21.61	24.8	27.73	64.95	4.29	8.63	16.95	32.76	40.71	45.32
		img: sep, map: shared [128m]	0.19	110.97	1.31	4.45	15.02	31.34	38.41	41.39	53.5	6.59	12.77	24.75	44.85	52.43	56.57
		img: sep, map: shared [**chain**]	0.23	107.06	1.99	6.23	18.26	33.49	39.56	42.39	53.25	6.7	13.34	25.9	45.94	53.17	56.41
	b	img: shared, map: shared [32m]	0.35	129.01	1.26	4.45	11.77	23.81	26.58	29.88	64.56	4.08	8.79	16.8	32.91	40.55	45.26
		img: shared, map: shared [128m]	0.21	108.08	1.31	4.6	15.75	34.43	40.55	43.75	53.15	7.22	13.66	25.8	45.32	52.33	55.36
		img: shared, map: shared [**chain**]	0.24	103.84	1.62	6.54	18.52	36	41.81	44.85	52.02	7.54	14.08	26.22	46.83	54.26	57.67
	c	2a. + weights [32m]	0.34	123	1.31	4.87	14.76	25.85	30.51	33.49	60.32	4.6	9.84	19.15	36.05	44.43	48.51
		2a. + weights [128m]	0.2	108.51	1.57	5.44	17.37	36.11	42.7	45.47	51.03	7.38	13.87	26.37	47.25	54.16	57.25
		2a. + weights [**chain**]	0.24	100.25	2.41	7.17	20.93	36.89	43.85	47.04	50.18	7.64	15.07	28.47	49.97	56.36	58.87
	d	2b. + weights [32m]	0.33	130.61	1.47	4.97	13.55	23.44	27.37	30.14	64.91	4.19	9.52	17.06	34.12	41.39	44.95
		2b. + weights [128m]	0.19	99.64	1.52	6.02	17.84	36.47	42.96	46.52	49.94	7.17	14.08	28.41	48.98	56.1	58.82
		2b. + weights [**chain**]	0.23	101.76	2.41	7.74	20.62	36.47	42.39	46.15	50.23	7.38	15.07	28.83	49.35	55.83	59.03
	e	img:shared + map:sep + weighted [32m]	0.31	130.6	1.73	5.08	16.12	25.64	29.57	32.76	59.87	4.4	8.95	18.11	35.01	43.43	48.61
		img:shared + map:sep + weighted [128m]	0.18	106	1.15	4.87	17.16	35.95	43.49	47.25	46.78	7.27	14.08	27	49.56	56.93	60.65
		img:shared + map:sep + weighted [**chain**]	0.22	**97.49**	**2.72**	**8.53**	**22.92**	**39.77**	**46.62**	**50.03**	**43.64**	**8.16**	**15.12**	**29.15**	**53.17**	**59.97**	**63.63**

Table 4.6: Localizing single-scale and multi-scale BEVS (Search Radius: 256m). Single-Scale: Increasing BEV depth up to 128m results in increasing recall rates, despite decreasing resolution. Chaining multiple single-scale models yields highest accuracies. Increasing resolution of input map raster is beneficial for coarser models. Multi-Scale: img: sep/shared indicates whether we separate/share decoders for fine and coarse features. map: sep/shared indicates whether we separate/share the encoders - the decoders are always separate. Shared features (2b) outperforms (2a). Uncertainty-weighted loss (2c and 2d) outperform simple loss (2a and 2b). Using separate map encoders with shared image features and separate map encoders consistently score the highest (2e).

Model Type	idx	Model	Uncertainty	XY Mean Error (m)	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	Yaw Mean Error (°)	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
1. Single-Scale	a	32m	0.27	4.87	2.51	10.2	29.04	60.28	86.81	100	18.56	10.05	18.94	36.21	68.5	83.73	88.23
	b	64m	0.2	4.59	2.41	9.68	28.26	63.53	90.06	100	16.67	10.88	21.09	39.93	73.21	85.71	89.38
	c	128m	0.15	4.7	1.73	5.91	20.62	65.1	90.63	100	11.4	12.24	22.87	42.86	76.03	88.23	92.94
	d	Chain(32m, 64m)	0.21	4.46	**3.24**	**11.67**	32.34	65.1	89.74	100	14.51	11.62	22.71	43.38	76.3	88.17	91.31
	e	Chain(64m,128m)	0.15	4.52	2.41	9.58	28.78	64.15	90.32	100	11.81	12.3	23.5	44.22	78.02	89.17	92.67
	f	Chain(32m,128m)	0.21	4.44	2.51	10.41	31.71	65.1	89.95	100	10.94	12.24	23.55	44.69	79.7	90.95	93.67
	g	Chain(32m, 64m, 128m)	0.18	**4.32**	3.14	11.04	**33.49**	**66.25**	90.53	100	**10.17**	**12.87**	**24.44**	**46.52**	**80.74**	**91.42**	**94.09**
	h	64m - Finer Map Input	0.19	4.48	2.62	10.94	31.76	63.27	90.01	100	13.55	11.36	21.82	41.55	75.61	87.7	91.68
	i	128m - Finer Map Input	0.13	4.58	1.88	7.69	26.22	63.37	**91.42**	100	13.26	11.41	22.34	42.8	76.66	87.96	92.46
2. Multi-Scale	a	img: sep, map: shared [32m]	0.27	4.74	2.77	9.79	29.04	61.64	89.06	100	17.59	9.26	18.37	34.75	69.13	83.99	89.27
		img: sep, map: shared [128m]	0.13	4.57	2.15	8.06	27	63.53	90.84	100	13.14	11.46	22.5	42.07	76.3	88.64	92.31
		img: sep, map: shared [**chain**]	0.2	4.4	2.3	10.36	30.61	65.15	91.58	100	11.55	12.14	23.65	43.8	79.07	90.69	93.67
	b	img: shared, map: shared [32m]	0.27	4.52	2.88	9.99	29.72	64.52	90.42	100	15.25	9.94	18.89	34.96	69.28	84.35	90.32
		img: shared, map: shared [128m]	0.14	4.44	2.04	7.8	27.52	66.04	91.84	100	11.6	12.56	23.44	42.96	77.66	89.9	93.41
		img: shared, map: shared [**chain**]	0.21	4.23	2.72	11.04	32.08	67.77	**92.26**	100	**10.15**	**13.24**	**24.59**	45.11	79.96	**91.31**	**94.51**
	c	2a. + weights [32m]	0.27	4.44	2.62	9.84	31.55	64.84	90.58	100	16.65	10.15	19.36	37.15	70.17	84.41	89.9
		2a. + weights [128m]	0.14	4.34	1.78	8.37	28.99	67.35	91.84	100	12.51	11.09	21.72	41.97	76.24	88.38	92.78
		2a. + weights [**chain**]	0.21	4.2	2.93	10.88	33.39	**68.6**	91.68	100	12	12.56	23.55	44.53	78.96	90.01	93.41
	d	2b. + weights [32m]	0.27	4.49	3.3	11.83	31.55	63.84	90.16	100	16.45	9.37	19.2	34.96	70.23	85.09	89.9
		2b. + weights [128m]	0.13	4.36	2.35	9.52	29.2	66.98	91.58	100	11.87	11.67	23.08	44.64	78.34	89.38	92.36
		2b. + weights [**chain**]	0.2	**4.18**	**3.98**	**11.93**	34.64	67.82	91.73	100	11.16	12.19	24.49	**45.79**	**80.69**	90.69	93.46
	e	img:shared + map:sep + weighted [32m]	0.25	4.6	3.19	10.99	32.76	62.79	89.01	100	16.43	9.79	19	36.16	70.33	84.51	89.69
		img:shared + map:sep + weighted [128m]	0.13	4.43	1.67	7.74	28.05	66.09	91.78	100	11.41	12.19	23.23	43.22	78.49	89.01	93.2
		img:shared + map:sep + weighted [**chain**]	0.19	4.25	3.72	11.62	**34.69**	66.41	91.31	100	10.92	12.66	23.81	44.27	80.22	90.69	93.72

Table 4.7: Localizing single-scale and multi-scale BEVS (Search Radius: 10m). Single-Scale: Increasing BEV depth ranges up to 128m results in increasing recall rates, despite decreasing resolution. Chaining multiple single-scale models yields highest accuracies. Increasing resolution of input map raster is beneficial for coarser models. Multi-Scale: Naming convention as in 4.6.

4.3 Design Decision: Multi-Scale Models

As we saw in the previous section, learning BEVs and maps at different resolutions offers great benefits for localization. However, we are more interested in learning whether a single model is capable of achieving similar results. In this section, we report experiments that we ran experiments in search for a single strong model that can retain the best of both worlds, i.e. obtain fine accuracies when localizing in a large search space.

As described in the Method section, all our multiscale models consist of two branches, each at a different resolution. From our single-scale experiments listed in Table 4.6 and 4.2.4, we identify the best pair to be the 32m and 128m. This provides a good balance between fine and coarse resolutions: one branch helps learn the finer features for precisely localizing with sub-meter errors, while the other learns coarser features to provide a good coarse localization in a large search space.

4.3.1 Neural BEV

For the two branches of our model, we consider two different ways of learning features from the images.

Separate Image Decoders. We modify our image feature extractor to have two different decoders, one that learns fine features, and the other coarse features. We also separate our scale classifiers, that are focused on predicting scales from only one type of image features. We then have two 3D grids that project onto their corresponding image features. This method benefits from extra capacity per task and allows learning different representations that are complementary, although it is difficult to say whether this is beneficial. To study this, we report results in 2a. in our results tables.

Shared Image Decoders. Different from the above, we hypothesize that it is more likely that both sets of features would benefit from being more interconnected. To achieve this, we choose to output a 256-dimension feature from our image extractor's decoder. Since we use an FPN decoder,

this implies that each level in our decoder has a 256-dimensional feature. Row 2b. in Tables 4.6 and 4.2.4 support this argument. Sharing features in the decoder results in consistently outperforming the previous model.

4.3.2 Neural Map Encoding

In the multiscale setup, we experiment with two methods.

We first use a single encoder that takes as input two map rasters of the same resolution. Two separate decoders then output two neural maps at different resolutions. Recall that during training, we require two different maps covering areas - one large and one small. We choose an input resolution of 1mpp (both maps). The first decoder outputs a neural map at 0.5mpp. This is done by adding an extra upsampling layer similar to the layers in the decoder. The second decoder outputs a neural map at 2mpp from its penultimate layer. The benefit of this method is that we use a single encoder backbone to process both maps. The downside of this method is that the first decoder's final upsampling layer is tasked with creating information that does not exist. Note that using an input of 0.5mpp is very computationally expensive in this case because the encoder will have to process a very large input map raster to produce the coarser neural map. Our experiments 2(a)-(d) use this method.

In the second method, we instead use two separate decoders. This allows both map encoders to learn separate features, also removes the need for an upsampling layer. We argue that there is little benefit from sharing an encoder. Our first map encoder processes 0.5mpp map rasters and outputs 0.5mpp neural maps. The second processes 1mpp map rasters and outputs 2mpp neural maps in the penultimate decoder layer. Recall that in Sec 4.2.3 and Row 1(i) in the tables, we saw that greatly boosts coarse neural map quality.

4.3.3 Loss

We now experiment with and without the uncertainty weighting. We run experiments 2(a) and 2(b) with uncertainty and see higher accuracies across all metrics. This is due to effectively balancing two task with varying difficulty levels during training. The optimal weights are then used for evaluation. After training, 2(c)'s 32m and 128m branch have scaling factors of $(1/\sigma^2_{32m})$ = 447.0, and $(1/\sigma^2_{128m})$=347.29 respectively. Similarly, 2(d)'s scaling factors are $(1/\sigma^2_{32m})$ = 451.24, $(1/\sigma^2_{128m})$=338.94. This indicates that the loss corresponding to the 32m branch is made peakier/more confident than the 128m branch. This agrees with our intuition and previous experiments: since the larger BEV results in lower uncertainty, the 128m branch's NLL loss is typically lower. Thus, in order to balance both losses, the lower loss is scaled up. A better balance directs the gradients towards more more optimal fine features, which subsequently improves localization.

4.4 Results

We fully train to convergence our best model from the previous section (see Row 2(e) in Tables 4.6 and 4.2.4). In Tables 4.4 (search radius = 256m) and 4.4 (search radius = 10m), we evaluate our method and compare it against OrienterNet. For completeness, we also evaluate our method on a 48m search radius in Table 4.4, which is what OrienterNet was originally trained and evaluated on.

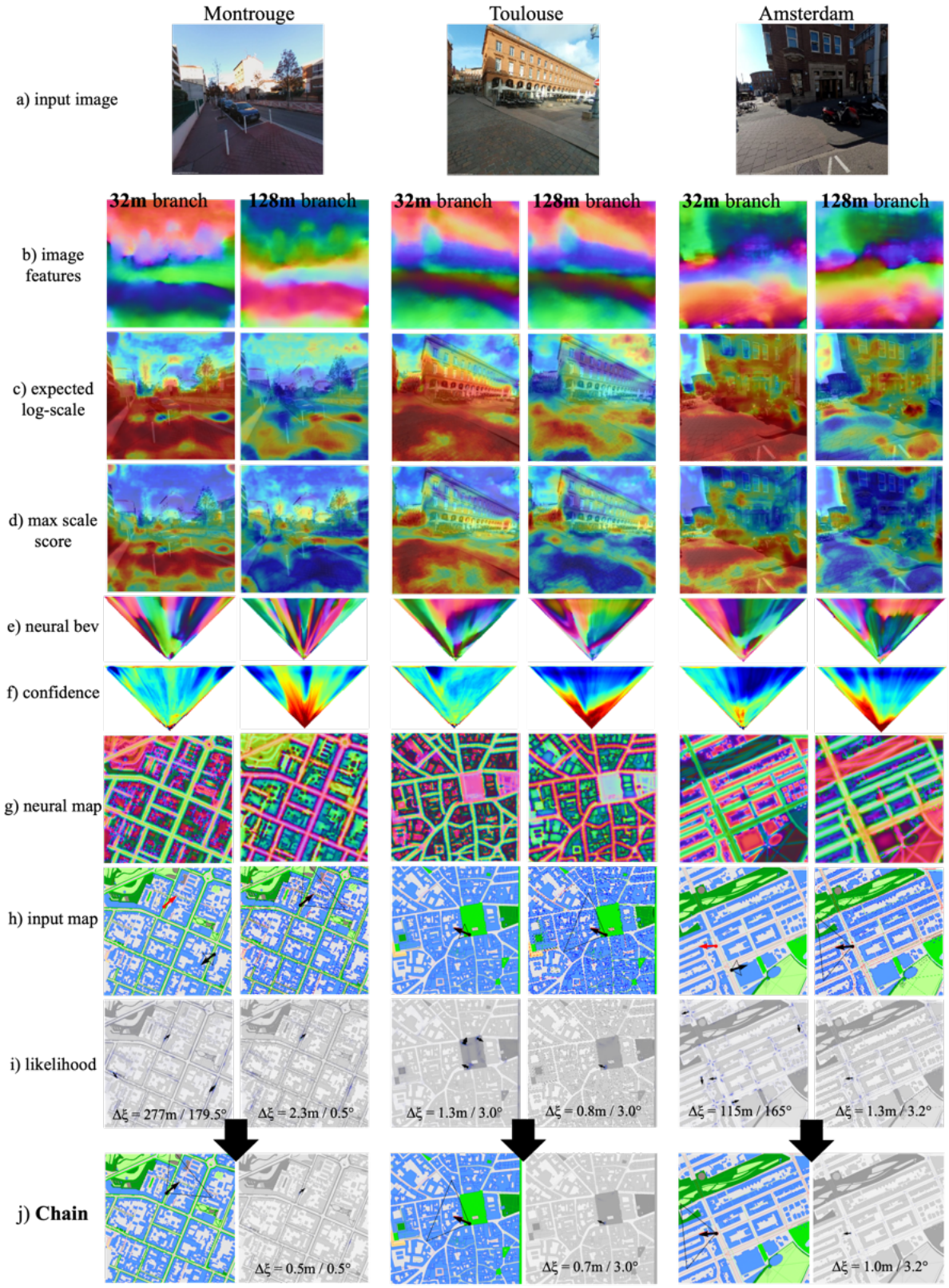

Figure 4.8: **Visualization of the outputs of the coarse and fine branches**. Row **(a)** shows 3 input test images. Rows **(b)**-**(f)** visualizes the predictions of the BEV mapping module. Row **(g)** contains the PCA plots of the neural map encodings of **(h)**. Row **(i)** shows the pose likelihoods per branch. Note the reduction in pose error when both likelihoods are chained in row **(j)**.

Figure 4.9: **OrienterNet vs Ours**. Our model learns an optimal combination of coarse and fine features covering larger distances (BEV outlines shown at the predicted pose) than OrienterNet. The examples above show OrienterNet's failures being successfully localized by our model. GT and Predicted poses are shown in red and **black** respectively.

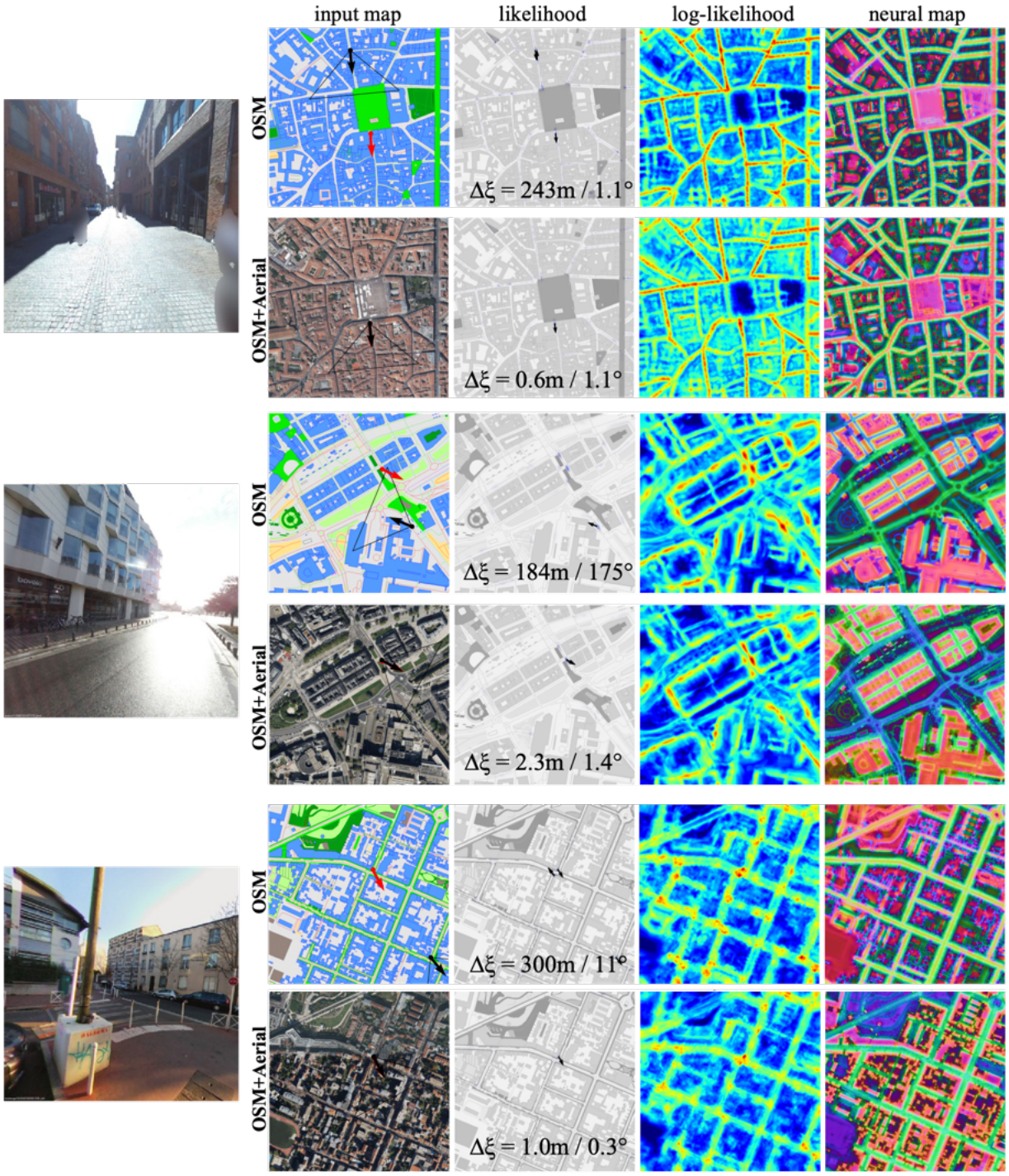

Figure 4.10: **OSM-only (Failure), OSM+Satellite (Success)**. We see OSM+Satellite successfully localizing these images which OSM-only cannot. This can be attributed to the road markings, trees, poles and other objects that are visible only on the satellite imagery.

Figure 4.11: **Non-Localizable Failure Cases**. These are examples of images that lack distinctive visual cues that could be used to correctly localize the camera.

Our method yields an increase of approximately 20% Recall under 20m in the 256m radius search area. In the 10m radius search area, our method obtains a lower error of 23cm/4.48° on average compared to OrienterNet.

In Figure 4.8, we demonstrate our model's capabilities and visualize various outputs of the network. In Figure 4.9, we show examples where our method outperforms OrienterNet.

experiment	uncertainty	xy mean error	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	yaw mean error	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
GPS	-	4.53	13.29	24.8	41.08	66.14	88.07	97.8	-	-	-	-	-	-	-
OrienterNet	-	109.58	2.83	9	22.92	34.38	38.41	41.23	52.79	6.54	12.3	23.29	42.86	50.81	54.74
Ours (32m branch)	0.33	109.5	2.15	8.11	20.83	32.86	37.57	40.87	55.6	6.12	11.25	22.19	41.81	49.92	53.43
Ours (128m branch)	0.18	78.94	2.56	7.8	23.5	45.79	54.42	57.51	38.16	9.21	17.53	33.96	57.98	65.67	69.13
Ours	0.22	71.14	3.35	11.67	29.98	48.82	56.62	60.07	37.42	9.47	17.53	34.9	60.02	67.66	70.02
Ours with Satellite (32m branch)	0.3	97.39	2.77	9.63	24.7	39.19	42.86	46.47	49.7	6.44	12.66	23.76	46.26	54.53	58.45
Ours with Satellite (128m branch)	0.18	68.52	2.98	8.84	27.32	50.13	58.97	61.9	35.08	9.79	18.42	35.06	61.17	69.34	71.9
Ours with Satellite	0.21	**64.15**	**3.66**	**13.4**	**33.07**	**53.53**	**60.65**	**64.1**	**33.2**	**10.26**	**19.36**	**36.42**	**63.58**	**70.91**	**73.42**

Table 4.8: Ours vs OrienterNet: Search Radius=256m (Retrieval)

experiment	uncertainty	xy error	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	yaw error	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
GPS	-	4.53	13.29	24.8	41.08	66.14	88.07	97.8	-	-	-	-	-	-	-
OrienterNet		4.06	**5.02**	15.65	39.72	69.07	91.05	**100**	12.58	11.25	21.04	40.03	75.82	88.8	92.67
Ours (32m branch)	0.26	4.11	3.87	14.91	37.47	68.29	91	**100**	12.98	11.2	20.36	39.87	74.1	88.96	92.67
Ours (128m branch)	0.13	3.95	3.77	11.36	34.64	70.96	93.56	**100**	8.11	13.24	25.01	46.89	81.79	92.78	95.55
Ours	0.2	3.83	4.24	16.06	40.55	71.27	92.83	**100**	8.5	**13.55**	**26.01**	**49.5**	**83.73**	**93.41**	95.5
Ours with Satellite (32m branch)	0.24	3.96	4.19	15.02	39.46	70.59	91.68	**100**	12.39	11.04	21.04	39.4	75.04	89.01	92.62
Ours with Satellite (128m branch)	0.13	3.86	3.04	11.04	34.22	72.84	**93.93**	**100**	8.03	12.56	24.12	45.37	81.63	93.04	95.87
Ours with Satellite	0.19	**3.63**	3.72	**16.33**	**42.7**	**74.73**	93.46	**100**	**8.02**	13.03	25.33	47.51	83.36	**93.41**	**96.02**

Table 4.9: Ours vs OrienterNet: Search Radius=10m (Precision)

experiment	uncertainty	xy error	R@0.5m	R@1m	R@2m	R@5m	R@10m	R@20m	yaw error	R@0.5°	R@1°	R@2°	R@5°	R@10°	R@20°
GPS	-	4.53	13.29	24.8	41.08	66.14	88.07	97.8	-	-	-	-	-	-	-
OrienterNet		15.09	4.19	13.13	33.28	55.15	65.83	74.73	27.29	9.73	17.69	34.8	65.52	77.13	80.48
Ours (32m branch)	0.28	15.66	3.19	12.51	31.19	53.74	64.26	73.52	28.79	9.47	16.8	33.44	63.16	75.09	79.12
Ours (128m branch)	0.15	11.51	2.93	10.05	31.4	60.54	74.57	82.37	18.46	12.14	22.4	42.49	73.78	84.51	86.76
Ours	0.21	11.33	4.03	14.7	36.63	62.01	74.57	82.31	18.62	12.09	**22.92**	**43.9**	75.67	85.14	87.23
Ours with Aerial (32m branch)	0.27	14.2	3.98	13.4	33.75	57.61	68.81	77.39	26.33	9.16	17.69	33.7	64.84	76.87	80.69
Ours with Aerial (128m branch)	0.15	11.09	2.98	10.99	31.34	63.47	76.92	83.83	16.84	**12.4**	22.66	42.7	75.61	85.35	88.23
Ours with Aerial	0.2	**10.7**	**4.97**	**14.91**	**38.51**	**65.15**	**77.45**	**84.51**	**16.61**	11.77	22.29	42.33	**75.82**	**85.66**	**88.38**

Table 4.10: Ours vs OrienterNet: Search Radius=48m

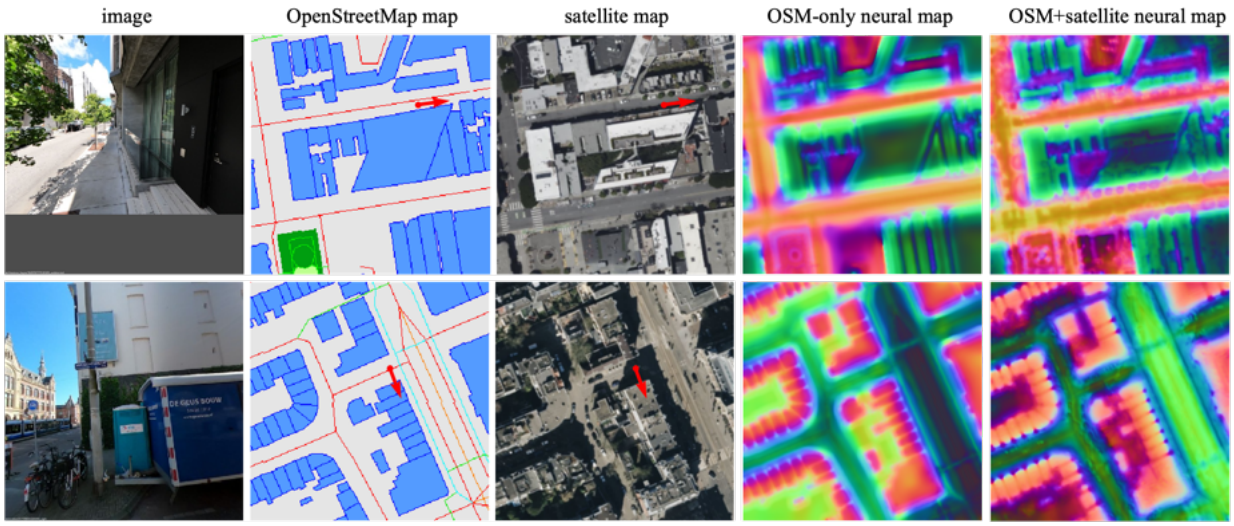

Figure 4.12: **Comparing Neural Maps**. Note the details that emerge when encodings of OSM and Satellite imagery are fused.

4.5 Fusing Satellite Imagery

In order to study the effect of training our multi-scale model with satellite imagery along with the OSM maps, we choose an early-fusion approach, where we first pass the RGB satellite map input through a 1x1 Convolutional layer to get a 16-dim embedding. This embedding is then concatenated with the OSM embeddings before passing it through the map encoder. In doing so, we allow the network to form strong relations between the two modalities at an early stage.

Our experiments show a 4% increase in Recall under 20m, so we fully train a model until convergence, with the identical training setup as our OSM-only fully trained model. Results in varying search spaces are shown in Tables 4.4, 4.4, and 4.4.

The main reason for the increase in accuracy is because of the extra information that comes with the satellite imagery. For example, OSM maps represent roads using lines to depict their centers. With satellite imagery, the network has access to the full width of roads and can thus correctly encode it within the neural maps. Moreover, the road markings and objects that are missing/unclassified in OSM like bollards that are otherwise visible on satellite imagery can now be encoded into the neural maps. Another advantage is that the network can learn to rely less on OSM for objects that may often be inaccurate (either due to mapping errors or changes in positioning over time), for example, poles or trees. We do not expect buildings visible in satellite imagery to provide any more information that those in OSM maps because only the rooftops are visible, and not their facades (which would be more useful for street-view image localization).

In Figure 4.12, we compare these neural maps with our OSM-only model.

Chapter 5

Discussion and Conclusion

5.1 Discussion

In this thesis, we break down OrienterNet [34] into its key components, which include (a) a *Bird's-Eye-View (BEV)* mapping from the image, (b) the *Neural Map* encoding of OpenStreetMap map tiles, and (c) the *Pose Estimation* of the BEV with respect to the Neural Map.

We study a forward (image to world) and inverse (world to image) mapping strategy (Sec. 4.2.1) and experimentally demonstrate the effectiveness of the latter for localization. Importantly, we report very strong improvements when we simply extend our BEV's depth range from 32m (as in OrienterNet) to 128m (Sec. 4.2.2), by trading away resolution to keep computational costs unchanged. Our results show that this yields higher retrieval scores when localizing on a large map, thanks to the useful information present at further distances from the camera. However, we observe a drop in accuracy when localizing in small search areas due to the decreased map resolutions (solved by our multi-scale model).

For learning coarse neural maps, we showed that we get better maps (indicated by the higher localization accuracies) by assigning the map downsampling task to the neural network, rather than the pre-processing. We experiment with different ways of downsampling and finally select a max-pool of the penultimate layers of our decoder network, with inputs having a 2x finer ground resolution (Sec. 4.3.2).

For pose estimation/matching, we show that OrienterNet's exhaustive matching implementation, despite scoring millions more poses than SNAP's RANSAC matching, requires drastically less memory and time while also being much more accurate than the RANSAC method. This is because of the Fast Fourier Transform implementation of the convolutional operation. We provide a comparative analysis in Section 4.5.

We combined all our learnings to propose a multi-scale model (Sec. 4.3) which is capable of gaining high retrieval scores and also finer accuracies, by jointly matching at different resolutions. Our model learns *complementary* fine and coarse features which together decide on a pose estimate. The joint learning is facilitated by predicting task uncertainties (Sec. 3.6), which account for the inherent difficulty faced by the finer branch that is responsible for localizing with only information within 32m.

Finally, we also propose an efficient method that additionally fuses satellite imagery (Fig. 4.10 to enable using richer features for localization. We observe richer neural maps that emerge from

this fusion (Fig. 4.12). Apart from localization, this novel combination of satellite imagery with OSM maps has great potential for future research in developing better ways of mapping our planet, similar to SNAP [35].

Our results show that our combination of improvements to OrienterNet together yields a roughly 20% increase in Recall under 20m. We gain an additional 4% improvement when using satellite imagery.

5.1.1 Limitations

Our method shares some of the limitations of OrienterNet. It assumes the camera intrinsics and gravity direction are known, which is not unreasonable as most mobile devices have sensors that expose this information.

Since our method has two branches for multiple scales, it requires longer training time and more memory. However, we carefully designed our method to keep this overhead at a minimum. Moreover, during inference, since our depth range is larger, we must infer on larger maps such that our 128m BEVs can be correctly matched at all (or sufficient) poses in the search space. While this entails a larger amount of memory and time requirement, we saw that this results in much higher accuracies. However, when speed is more important, our inference can easily be run in a hierarchical fashion - where the coarse branch first predicts likely poses and the fine branch only searches in the selected areas - resulting in significantly lower memory usage at the cost of a slight drop in accuracy (dictated by the coarse branch's retrieval quality).

Our model's BEVs and maps do not appear to be as interpretable as the maps of SNAP, which is potentially due to stronger supervision through the multi-view (from multiple street-view images) setup in SNAP. Also, since SNAP operates at a resolution of 20cm, and learns to differentiate between very similar poses, it is forced to learn a sharper map that encodes trees and road markings with higher positional accuracy.

5.1.2 Future Work

We identify four key areas of future work:

(a) Our method could benefit from BEV mapping strategies that more efficiently map the 3D world, and perhaps learn better depth estimates. For example, a scene representation that uses Gaussian splatting [22] could reduce the need for redundant points in free space (e.g. in the sky) and consequently learn the scene occupancy more efficiently. Our inverse mapping method computes scores for a uniform grid of 3D points without considering the contents of the image. An image with large occlusions such as a wall still requires computing a 3D grid volume that covers large areas behind the wall for inferring BEV maps.

(b) Furthermore, there is room for improvement in the quality of the BEV. Currently, objects and their boundaries are not easily distinguishable/interpretable in the BEV maps, which limits their usage for other real-world robotics applications. One potential direction is to additionally use explicit supervision for the BEV with the learned neural map. This could be a simple weighted L1 loss that encourages the network to learn BEVs that appear closer to the OSM map. This might also indirectly strengthen the depth supervision: a wrong depth estimate leads to a wrong BEV (eg. building wall positioned nearer/further), leading to a higher L1 loss and stronger gradients towards learning better depth/scales.

By using multiple co-visible images in a batch, we could also project points of one occupancy volume to another image plane for much stronger depth and feature supervision.

(c) Our output pose estimates can be integrated with Structure-from-Motion pipelines in multiple interesting ways. For example, they can serve as additional constraints in the form of pose priors for the bundle adjustment procedure. Differently, they can be used to geo-register SfM models (and their images), with the help of a RANSAC implementation. This enables recovering scale (which is unknown for SfM models) and world coordinates of any 3D model when the original city/neighbourhood is known.

(d) Finally, our method only estimates the 3-DoF pose (xy position and yaw) as we assume that the gravity direction is known. Future work could investigate relaxing this assumption by jointly estimating the roll and the pitch of the camera. This would give us a 5-DoF pose estimate, leaving only the camera height unknown. To extend to a 6-DoF problem, our method would need access to another source of data such as terrain data for supervision.

Bibliography

[1] Relja Arandjelovic, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. Netvlad: Cnn architecture for weakly supervised place recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2016.

[2] Vassileios Balntas, Shuda Li, and Victor Prisacariu. Relocnet: Continuous metric learning relocalisation using neural nets. In Proceedings of the European Conference on Computer Vision (ECCV), September 2018.

[3] Loick Chambon, Eloi Zablocki, Mickaël Chen, Florent Bartoccioni, Patrick Pérez, and Matthieu Cord. Pointbev: A sparse approach for bev predictions. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 15195–15204, June 2024.

[4] Changan Chen, Rui Wang, Christoph Vogel, and Marc Pollefeys. F3loc: Fusion and filtering for floorplan localization. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 18029–18038, June 2024.

[5] Wei Chen, Yu Liu, Weiping Wang, Erwin M Bakker, Theodoros Georgiou, Paul Fieguth, Li Liu, and Michael S Lew. Deep learning for instance retrieval: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(6):7270–7292, 2022.

[6] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops, pages 224–236, 2018.

[7] Mingyu Ding, Zhe Wang, Jiankai Sun, Jianping Shi, and Ping Luo. Camnet: Coarse-to-fine retrieval for camera re-localization. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), October 2019.

[8] Mihai Dusmanu, Ignacio Rocco, Tomas Pajdla, Marc Pollefeys, Josef Sivic, Akihiko Torii, and Torsten Sattler. D2-net: A trainable cnn for joint description and detection of local features. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2019.

[9] Florian Fervers, Sebastian Bullinger, Christoph Bodensteiner, Michael Arens, and Rainer Stiefelhagen. C-bev: Contrastive bird's eye view training for cross-view image retrieval and 3-dof pose estimation, 2023.

[10] Florian Fervers, Sebastian Bullinger, Christoph Bodensteiner, Michael Arens, and Rainer Stiefelhagen. Uncertainty-aware vision-based metric cross-view geolocalization. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 21621–21631, June 2023.

[11] Jan-Michael Frahm, Pierre Fite-Georgel, David Gallup, Tim Johnson, Rahul Raguram, Changchang Wu, Yi-Hung Jen, Enrique Dunn, Brian Clipp, Svetlana Lazebnik, et al. Building rome on a cloudless day. In Computer Vision–ECCV 2010: 11th European Conference on Computer Vision, Heraklion, Crete, Greece, September 5-11, 2010, Proceedings, Part IV 11, pages 368–381. Springer, 2010.

[12] Dorian Galvez-López and Juan D. Tardos. Bags of binary words for fast place recognition in image sequences. IEEE Transactions on Robotics, 28(5):1188–1197, 2012.

[13] Adam W Harley, Zhaoyuan Fang, Jie Li, Rares Ambrus, and Katerina Fragkiadaki. Simple-bev: What really matters for multi-sensor bev perception? In 2023 IEEE International Conference on Robotics and Automation (ICRA), pages 2759–2765. IEEE, 2023.

[14] Henry Howard-Jenkins and Victor Adrian Prisacariu. Lalaloc++: Global floor plan comprehension for layout localisation in unvisited environments. In European Conference on Computer Vision, pages 693–709. Springer, 2022.

[15] Henry Howard-Jenkins, Jose-Raul Ruiz-Sarmiento, and Victor Adrian Prisacariu. Lalaloc: Latent layout localisation in dynamic, unvisited environments. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 10107–10116, October 2021.

[16] Anthony Hu, Zak Murez, Nikhil Mohan, Sofía Dudas, Jeffrey Hawke, Vijay Badrinarayanan, Roberto Cipolla, and Alex Kendall. Fiery: Future instance prediction in bird's-eye view from surround monocular cameras. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 15273–15282, October 2021.

[17] Sixing Hu, Mengdan Feng, Rang MH Nguyen, and Gim Hee Lee. Cvm-net: Cross-view matching network for image-based ground-to-aerial geo-localization. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 7258–7267, 2018.

[18] Junjie Huang, Guan Huang, Zheng Zhu, Yun Ye, and Dalong Du. Bevdet: High-performance multi-camera 3d object detection in bird-eye-view. arXiv preprint arXiv:2112.11790, 2021.

[19] Arnold Irschara, Christopher Zach, Jan-Michael Frahm, and Horst Bischof. From structure-from-motion point clouds to fast location recognition. In 2009 IEEE Conference on Computer Vision and Pattern Recognition, pages 2599–2606. IEEE, 2009.

[20] Linyi Jin, Jianming Zhang, Yannick Hold-Geoffroy, Oliver Wang, Kevin Matzen, Matthew Sticha, and David F. Fouhey. Perspective fields for single image camera calibration. In CVPR, 2023.

[21] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.

[22] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. ACM Trans. Graph., 42(4):139–1, 2023.

[23] Ted Lentsch, Zimin Xia, Holger Caesar, and Julian FP Kooij. Slicematch: Geometry-guided aggregation for cross-view pose estimation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 17225–17234, 2023.

[24] Philipp Lindenberger, Paul-Edouard Sarlin, Viktor Larsson, and Marc Pollefeys. Pixel-perfect structure-from-motion with featuremetric refinement. In Proceedings of the IEEE/CVF international conference on computer vision, pages 5987–5997, 2021.

[25] Philipp Lindenberger, Paul-Edouard Sarlin, Viktor Larsson, and Marc Pollefeys. Pixel-perfect structure-from-motion with featuremetric refinement. In Proceedings of the IEEE/CVF international conference on computer vision, pages 5987–5997, 2021.

[26] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. Lightglue: Local feature matching at light speed. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 17627–17638, 2023.

[27] Manuel Lopez, Roger Mari, Pau Gargallo, Yubin Kuang, Javier Gonzalez-Jimenez, and Gloria Haro. Deep single image camera calibration with radial distortion. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2019.

[28] Bing Maps. Bing maps. https://docs.microsoft.com/en-us/bingmaps.

[29] Zhixiang Min, Naji Khosravan, Zachary Bessinger, Manjunath Narayana, Sing Bing Kang, Enrique Dunn, and Ivaylo Boyadzhiev. Laser: Latent space rendering for 2d visual localization. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 11122–11131, June 2022.

[30] OpenStreetMap contributors. Planet dump retrieved from https://planet.osm.org . https://www.openstreetmap.org, 2017.

[31] Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16, pages 194–210. Springer, 2020.

[32] Avishkar Saha, Oscar Mendez, Chris Russell, and Richard Bowden. Translating images into maps. In 2022 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2022.

[33] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 4938–4947, 2020.

[34] Paul-Edouard Sarlin, Daniel DeTone, Tsun-Yi Yang, Armen Avetisyan, Julian Straub, Tomasz Malisiewicz, Samuel Rota Bulo, Richard Newcombe, Peter Kontschieder, and Vasileios Balntas. OrienterNet: Visual Localization in 2D Public Maps with Neural Matching. In CVPR, 2023.

[35] Paul-Edouard Sarlin, Eduard Trulls, Marc Pollefeys, Jan Hosang, and Simon Lynen. SNAP: Self-Supervised Neural Maps for Visual Positioning and Semantic Understanding. In NeurIPS, 2023.

[36] Paul-Edouard Sarlin, Ajaykumar Unagar, Mans Larsson, Hugo Germain, Carl Toft, Viktor Larsson, Marc Pollefeys, Vincent Lepetit, Lars Hammarstrand, Fredrik Kahl, et al. Back to the feature: Learning robust camera localization from pixels to pose. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 3247–3257, 2021.

[37] Grant Schindler, Matthew Brown, and Richard Szeliski. City-scale location recognition. In 2007 IEEE Conference on Computer Vision and Pattern Recognition, pages 1–7. IEEE, 2007.

[38] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 4104–4113, 2016.

[39] P Sermanet. Overfeat: Integrated recognition, localization and detection using convolutional networks. arXiv preprint arXiv:1312.6229, 2013.

[40] Shihao Shao, Kaifeng Chen, Arjun Karpur, Qinghua Cui, André Araujo, and Bingyi Cao. Global features are all you need for image retrieval and reranking. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 11036–11046, October 2023.

[41] Yujiao Shi and Hongdong Li. Beyond cross-view image retrieval: Highly accurate vehicle localization using satellite image. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 17010–17020, 2022.

[42] Yujiao Shi, Liu Liu, Xin Yu, and Hongdong Li. Spatial-aware feature aggregation for image based cross-view geo-localization. Advances in Neural Information Processing Systems, 32, 2019.

[43] Noah Snavely, Steven M Seitz, and Richard Szeliski. Photo tourism: exploring photo collections in 3d. In ACM siggraph 2006 papers, pages 835–846. 2006.

[44] Marvin Teichmann, Michael Weber, Marius Zoellner, Roberto Cipolla, and Raquel Urtasun. Multinet: Real-time joint semantic reasoning for autonomous driving. In 2018 IEEE intelligent vehicles symposium (IV), pages 1013–1020. IEEE, 2018.

[45] Akihiko Torii, Relja Arandjelovic, Josef Sivic, Masatoshi Okutomi, and Tomas Pajdla. 24/7 place recognition by view synthesis. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2015.

[46] A Vaswani. Attention is all you need. Advances in Neural Information Processing Systems, 2017.

[47] Alexander Veicht, Paul-Edouard Sarlin, Philipp Lindenberger, and Marc Pollefeys. GeoCalib: Single-image Calibration with Geometric Optimization. In ECCV, 2024.

[48] Lukas Von Stumberg, Patrick Wenzel, Qadeer Khan, and Daniel Cremers. Gn-net: The gauss-newton loss for multi-weather relocalization. IEEE Robotics and Automation Letters, 5(2):890–897, 2020.

[49] Xiaolong Wang, Runsen Xu, Zhuofan Cui, Zeyu Wan, and Yu Zhang. Fine-grained cross-view geo-localization using a correlation-aware homography estimator. Advances in Neural Information Processing Systems, 36, 2024.

[50] Kwang Moo Yi, Eduard Trulls, Vincent Lepetit, and Pascal Fua. Lift: Learned invariant feature transform. In Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part VI 14, pages 467–483. Springer, 2016.

[51] Kwang Moo Yi, Eduard Trulls, Yuki Ono, Vincent Lepetit, Mathieu Salzmann, and Pascal Fua. Learning to find good correspondences. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 2666–2674, 2018.